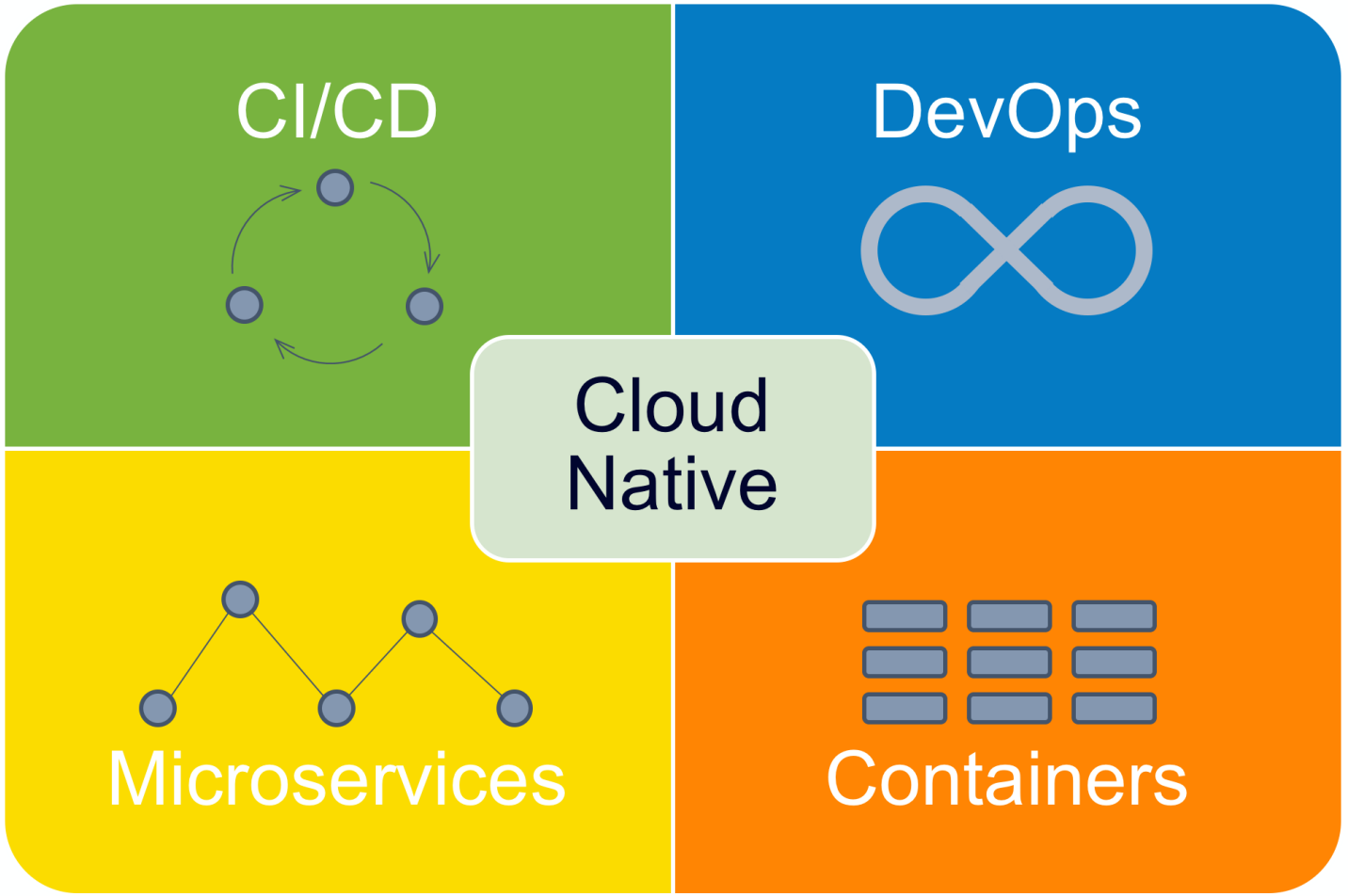
Briques de base du Cloud Native



Présenté par :
Elies Jebri

V2 03-22

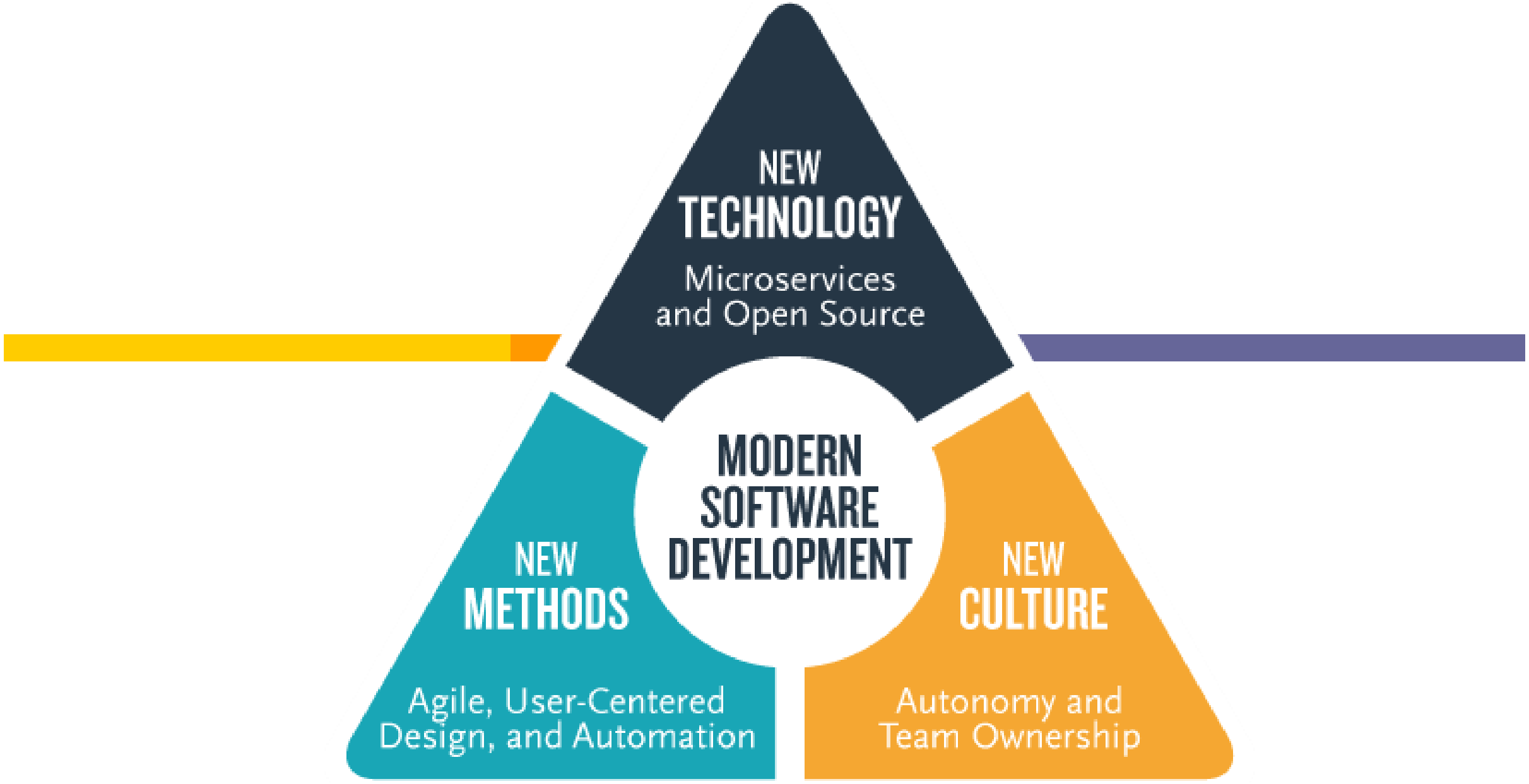
Elies Jebri : elies.jebri@gmail.com

Elies Jebri

- **Titre**: Consultant expert en OSS et infrastructures IT.
- **Distinctions & expérience**:
  - Expert depuis 2000 auprès d'un grand nombre de companies locales et internationales.
- **Formation**:
  - Agrégé en informatique industrielle,
  - Certifié RHCE, LPIC-1,2,301/303/304, NCLA, MCITP, CCNA, CEH, TenStep, AWS Practitioner, AWS SysOps, VMware Foundations, VMware Data Center Professional, Vmware Certified Instructor, Huawei Cloud Services, LPI-DevOps, DevOps Generalist, Cisco ENCOR, DCCOR
- **Contact**:
  elies.jebri@gmail.com

Elies Jebri

Elies Jebri : elies.jebri@gmail.com


NEW
TECHNOLOGY
Microservices
and Open Source
MODERN
SOFTWARE
DEVELOPMENT
NEW
METHODS
Agile, User-Centered
Design, and Automation
NEW
CULTURE
Autonomy and
Team Ownership

Comment DevOps affecte l'Architecture

- Aspects de l'architecture logicielle
- Terminologie de base et objectifs
- Anti-modèles, tels que le monolithe
- Le principe fondamental de la séparation des préoccupations
- Systèmes à trois tiers et microservices
- Nous terminerons enfin par quelques questions pratiques concernant migration de la base de données.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Plan

- Rappel Applications monolithiques vs Les architectures en Tier
- Les architectures Web
- Les microservices
- Les APIs REST

Elies Jebri

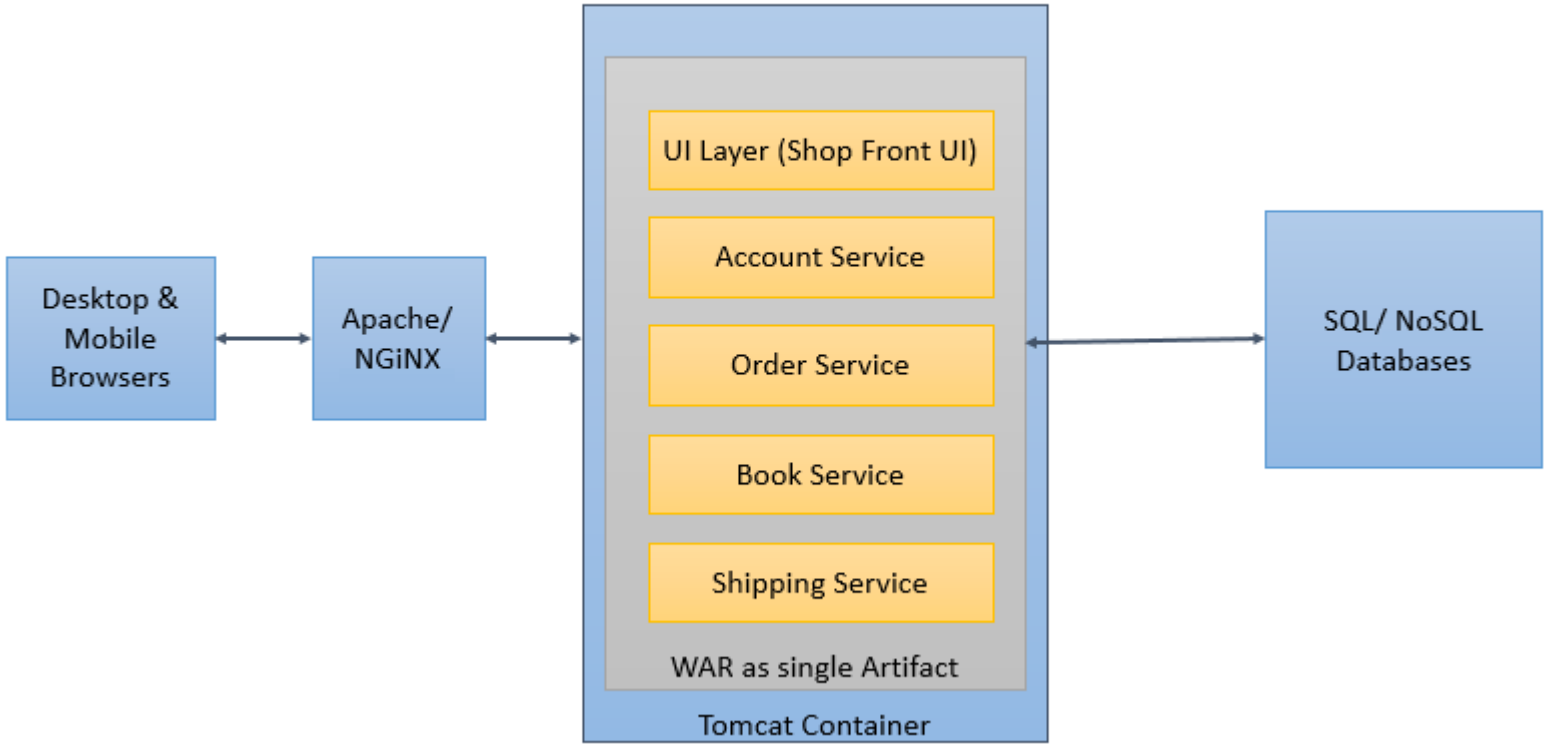
Elies Jebri : elies.jebri@gmail.com

ARCHITECTURES EN TIER

Elies Jebri : elies.jebri@gmail.com

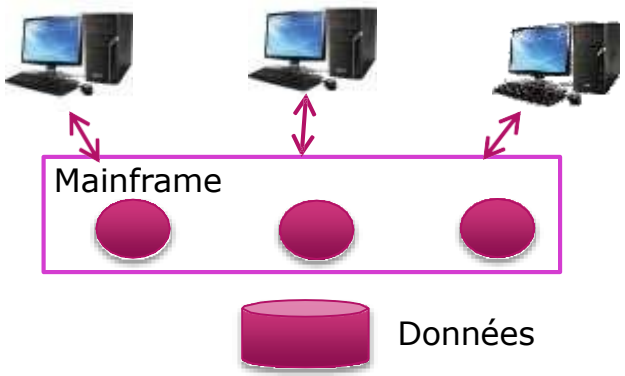
Application monolithique : rappel

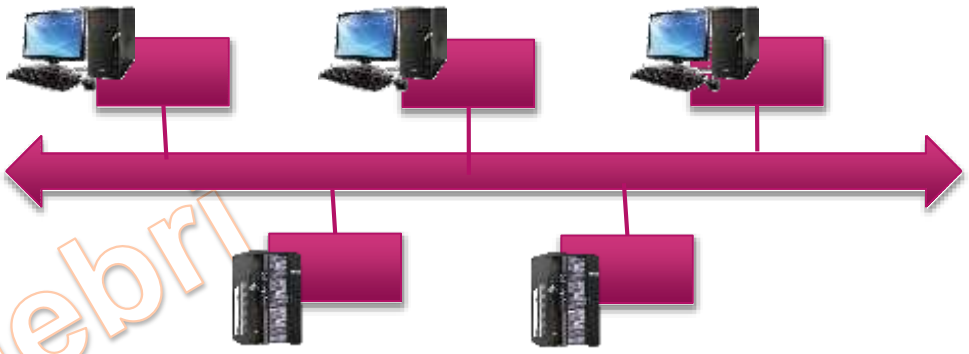
- Un gros code contenant toutes les fonctionnalités et les différentes couches logicielles
- Une seule grosse compilation et un seul livrable (un gros fichier WAR)
- Une seule pile logicielle (Linux, JVM, Tomcat et bibliothèques tierces)



Elies Jebri : elies.j

Du modèle centralisé au client-serveur





Architecture Centralisée	Architecture Répartie
Approche type Mainframe	PC, bureautique, client-serveur
Contrôle renforcé des données et de la logique métier	Peu de contrôle sur les données, très peu sur la logique métier
Centralisation des choix, des mises à jour	Problème de choix, installation et mise à jour des applications
Coût d'administration faible	Coûts d'administration par utilisateur élevé
Faible autonomie de l'utilisateur	Très (trop?) grande autonomie de l'utilisateur

Elies Jebri : elies.jebri@gmail.com

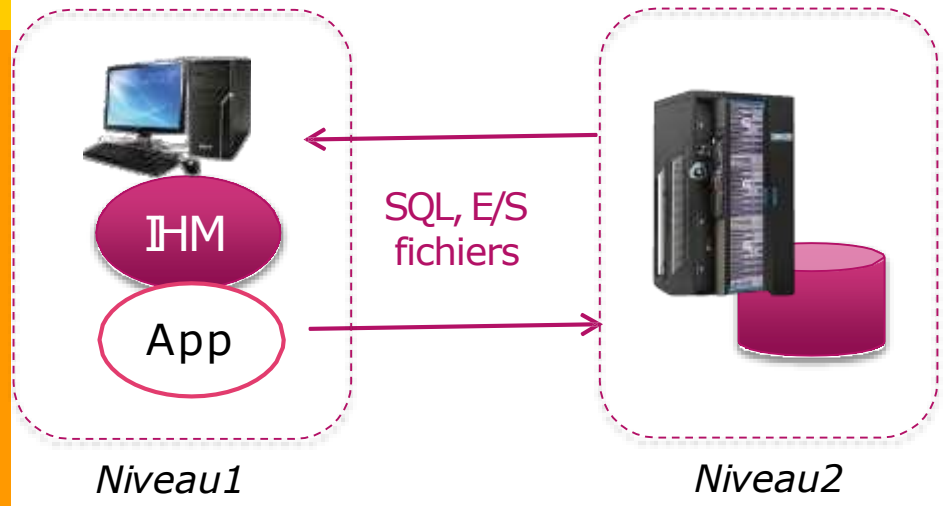
Architectures en Tier

- Principe : séparation des niveaux fonctionnels
  - IHM : interface Homme-Machine
  - Application métier
  - Gestion des données
- Le lien entre les niveaux est défini et limité à des interfaces
- Les applications peuvent être déployées et administrées de manière indépendante des IHM
- Placement des serveurs logiciels sur un ou plusieurs serveurs physiques
- Nombreuses variantes architecturales possibles

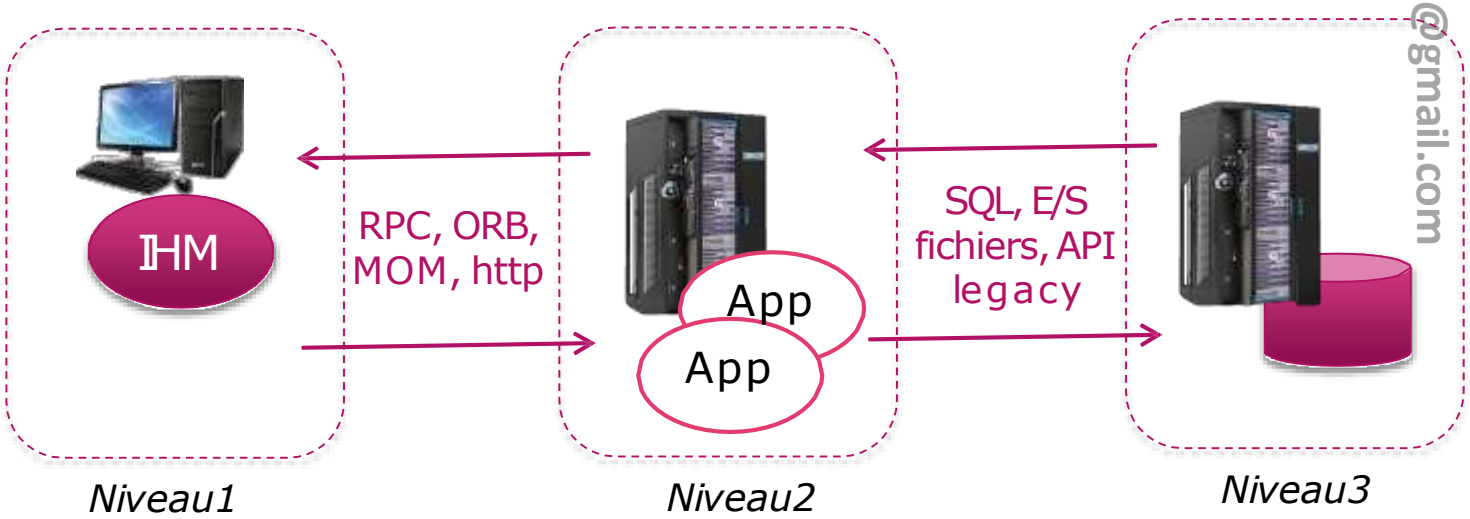
Elies Jebri

Elies Jebri : elies.jebri@gmail.com

De 2 à 3 Niveaux



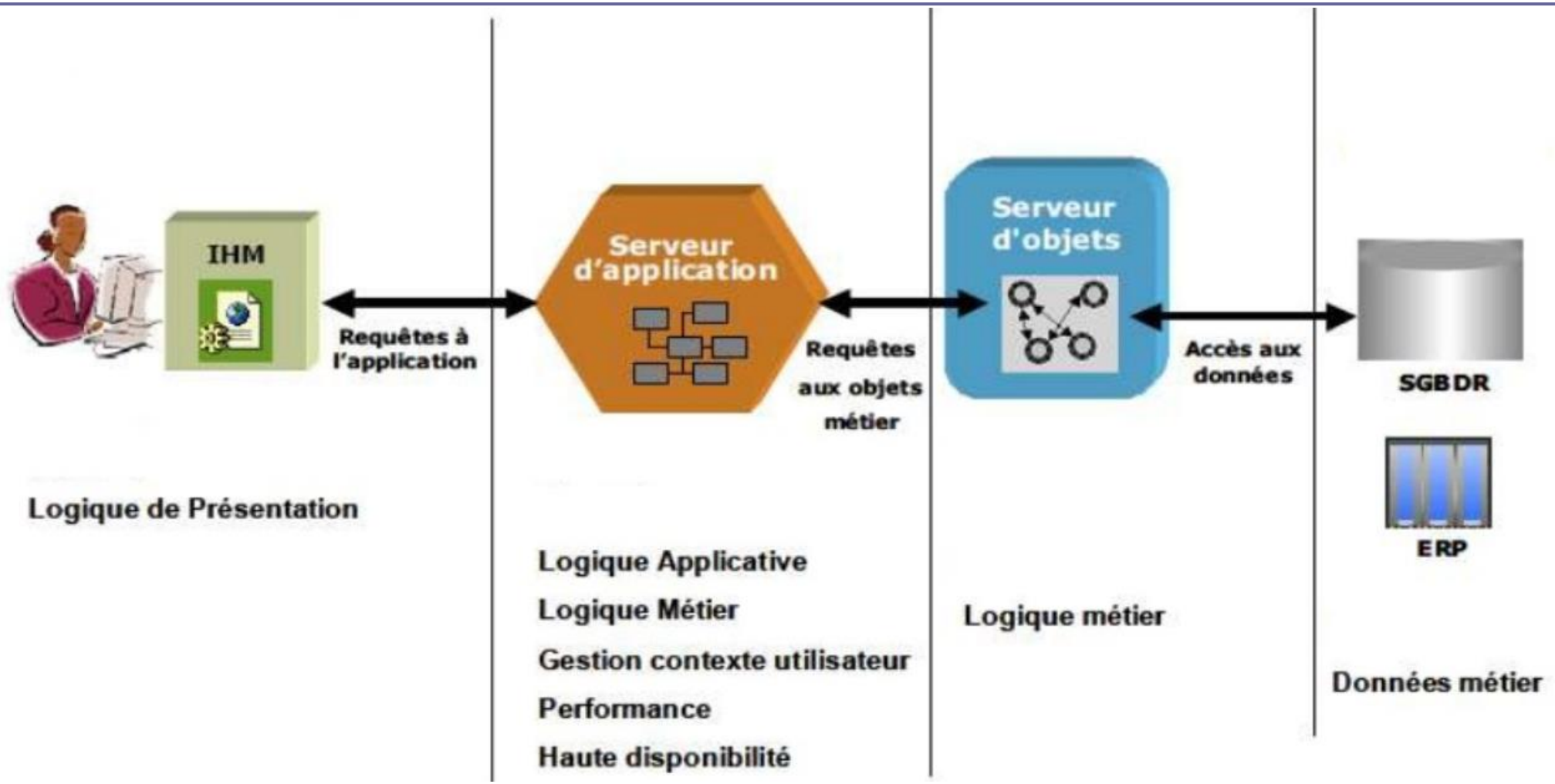
Client-Serveur à deux Niveaux



Client-Serveur à trois Niveaux

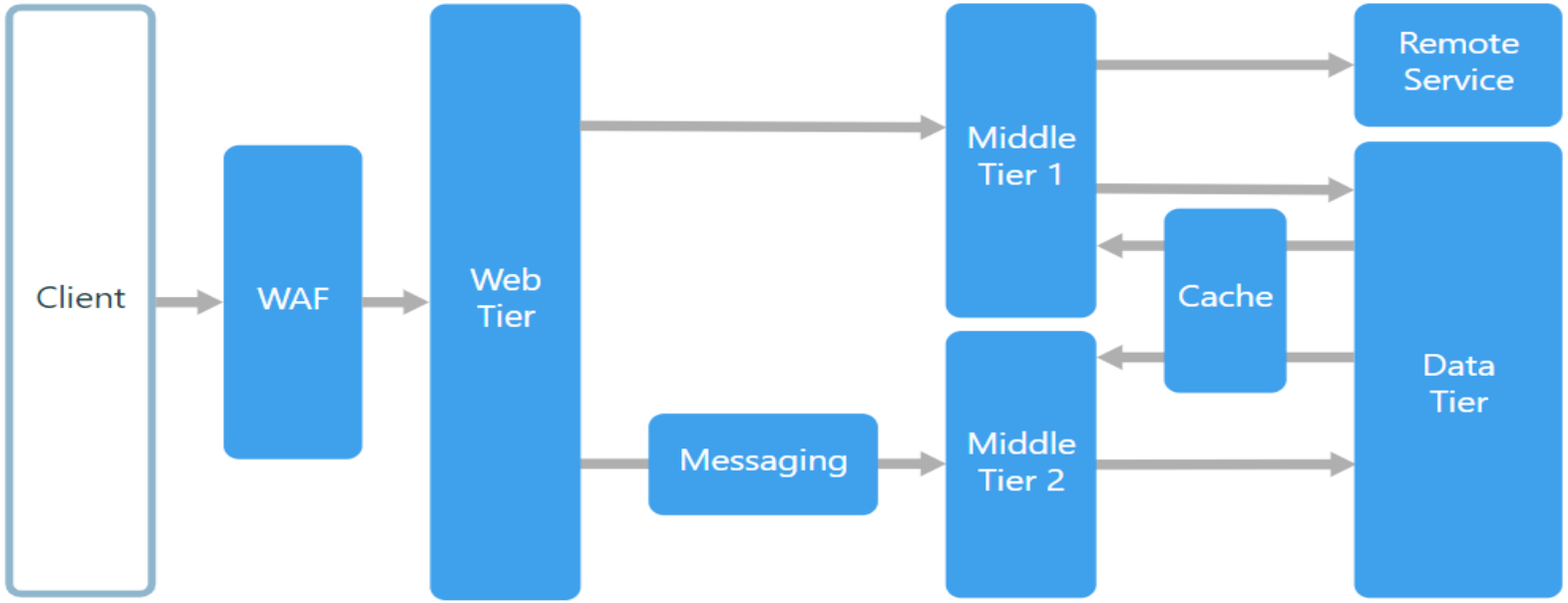
Elies Jebri : elies.jebri@gmail.com

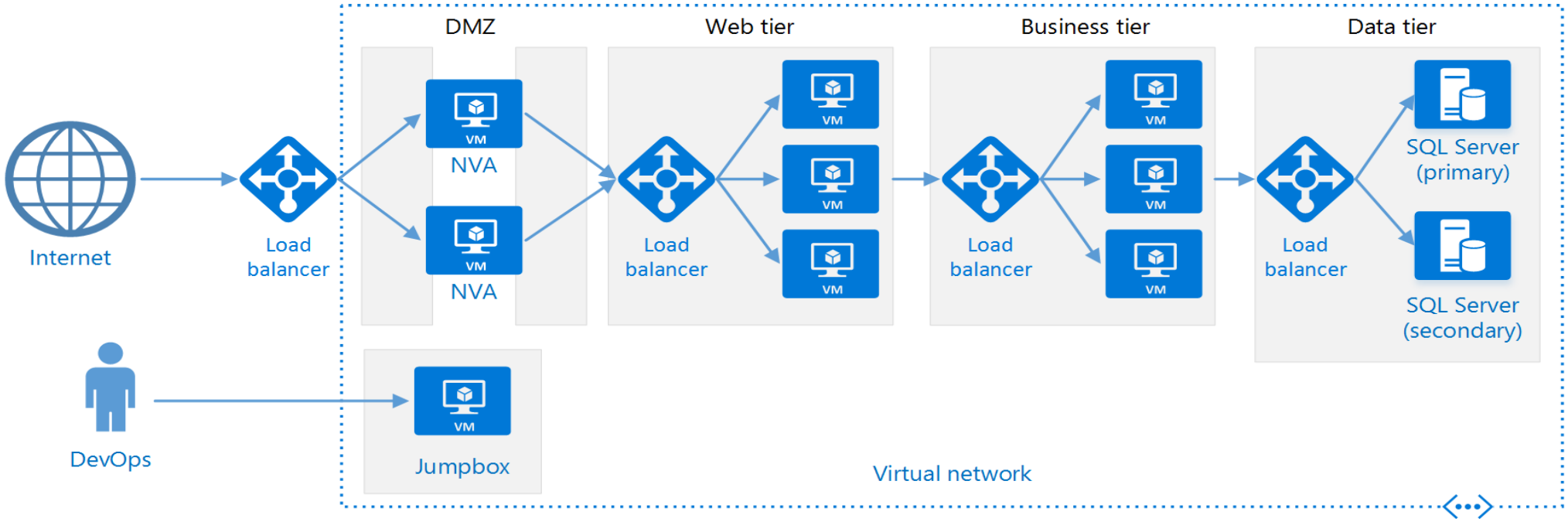
Architecture Multi-Tier



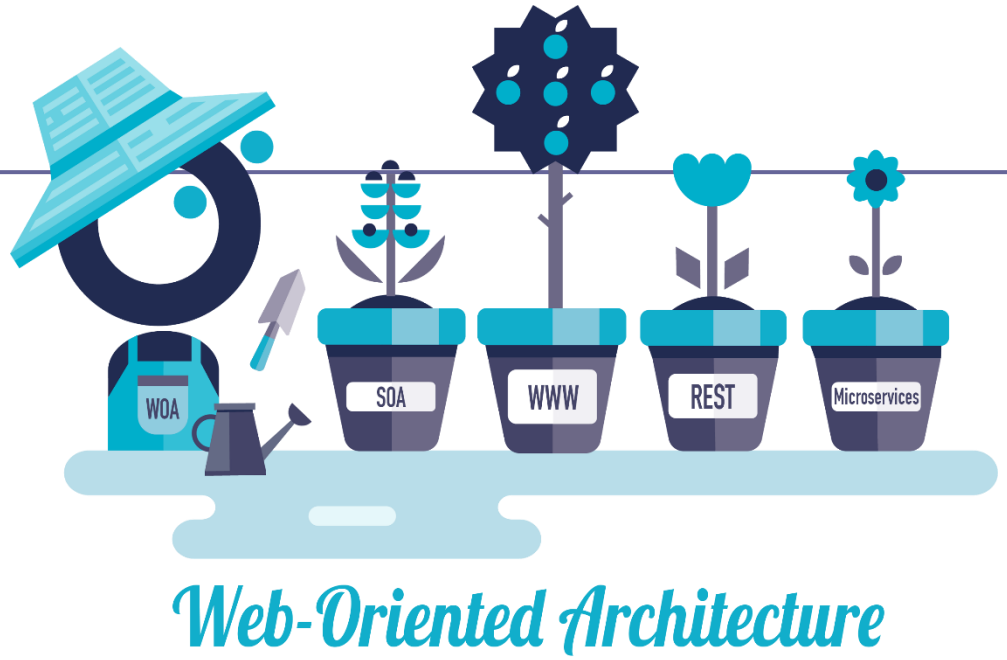
Elies Jebri : elies.jebri@gmail.com

Exemples d'architectures Multi-Tier





Elies Jebri : elies.jebri@gmail.com



LES ARCHITECTURES WEB

Elies Jebri : elies.jebri@gmail.com

Les architectures web

- Interconnexion de réseaux à l'échelle mondiale fondée sur les protocoles TCP/IP
- Évolution du client-serveur en local vers le client serveur sur Internet
- Modèle du « client léger », mode « non connecté »
- Exploitation des technologies internet dans le modèle client-serveur

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Les Web Services

- Unité logique applicative accessible en utilisant les protocoles standards d'internet, réutilisable et indépendante de la plateforme et de l'implémentation
- Objectifs
  - Accès rapide à l'information
  - Système ouvert réduisant les coûts
  - Administration simplifiée
  - Utilisation d'internet comme support de communication

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

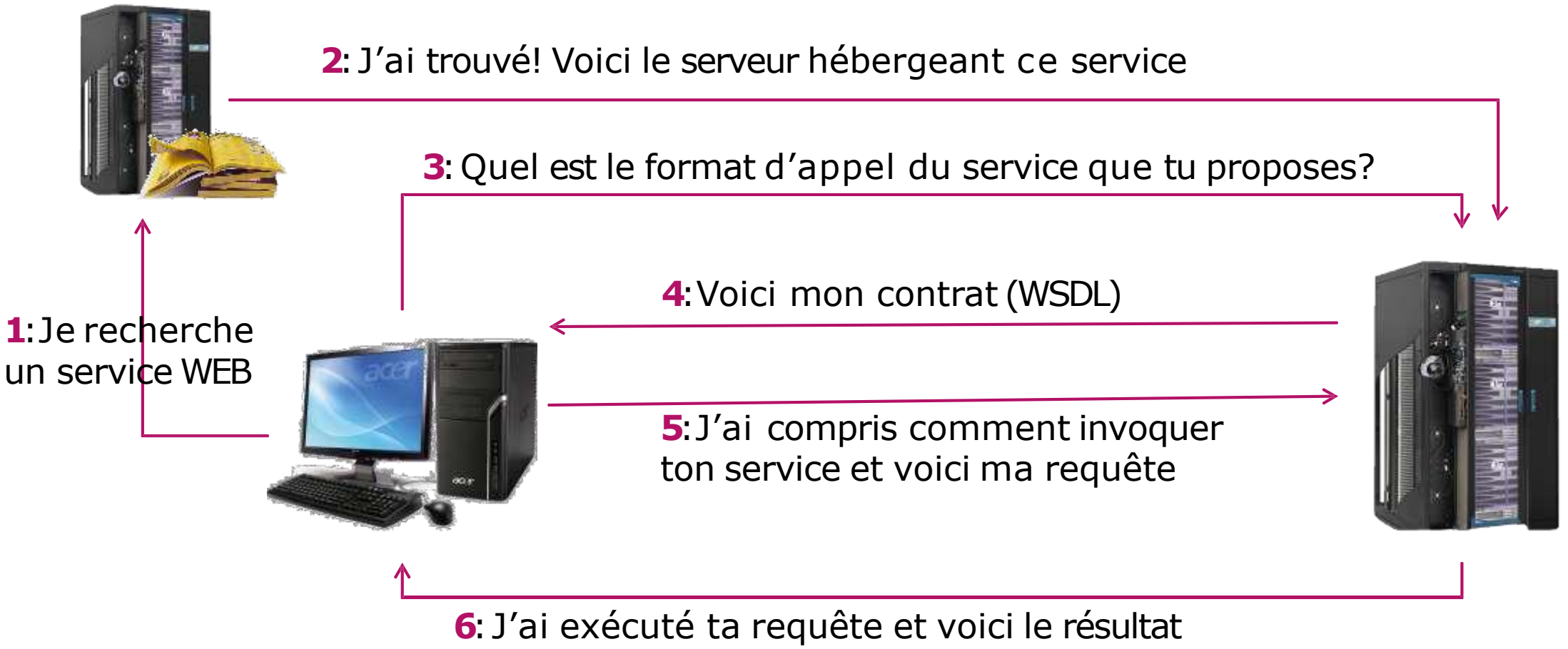
Caractéristiques des ServicesWeb

- Application fournissant des traitements et des données à d'autres applications déployées sur le Web et vues à travers une interface
- Protocole **SOAP** (Simple Object Access Protocol) over **HTTP**
- Données en **XML, JSON**
- Annuaire de services : **UDDI** (Universal Description Discovery and Integration)
- Langage **WSDL** (Web Service Description Language) pour la description du service

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Les Web Services :Principes



Elies Jebri : elies.jebri@gmail.com

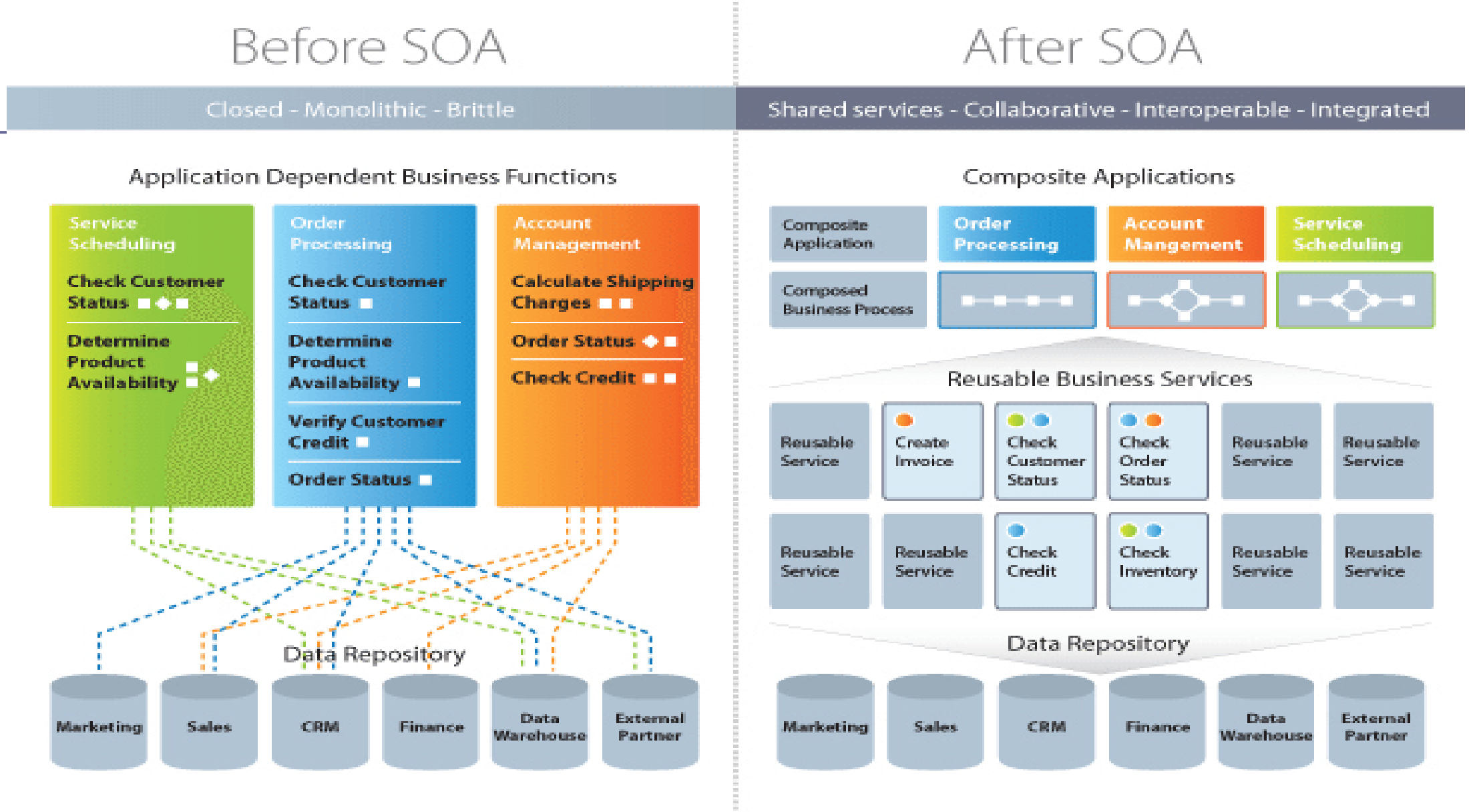
Les Architectures Orientées Services SOA

- Tout Devient «Service»…
- Service :
  - Fonctionnalité réutilisable dont le comportement est défini de façon contractuelle
- 3 Acteurs
  - **Consommateur de service** décrit le service à consommer
  - **Fournisseur de services** découvert en temps d'exécution à l'aide d'un intermédiaire (Registre de service)
  - **Registre de service** contient l'ensemble des descripteurs de services et les références vers les fournisseurs de services

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

En résumé


Before SOA
Closed - Monolithic - Brittle
Application Dependent Business Functions
Service Scheduling
Check Customer Status
Determine Product Availability
Order Processing
Check Customer Status
Determine Product Availability
Verify Customer Credit
Order Status
Account Management
Calculate Shipping Charges
Order Status
Check Credit
Data Repository
Marketing
Sales
CRM
Finance
Data Warehouse
External Partner
After SOA
Shared services - Collaborative - Interoperable - Integrated
Composite Applications
Composite Application
Order Processing
Account Mangement
Service Scheduling
Composed Business Process
Reusable Business Services
Reusable Service
Create Invoice
Check Customer Status
Check Order Status
Reusable Service
Reusable Service
Reusable Service
Reusable Service
Check Credit
Check Inventory
Reusable Service
Reusable Service
Data Repository
Marketing
Sales
CRM
Finance
Data Warehouse
External Partner

Architecture

ARCHITECTURE MICROSERVICES

Elies Jebri : elies.jebri@gmail.com

Les problématiques rencontrées par les entreprises

Problème n°1 :

- Comment faire en sorte que les applications proposées en ligne soient **toujours disponibles** et ne souffrent jamais de coupure ou de ralentissement, quelle que soit l'affluence des utilisateurs sur celles-ci ?
- Une solution est d'utiliser le **cloud**.
- Il faut donc envisager, dès le départ, une autre approche dans la conception de l'application afin que celle-ci soit "**Cloud-native**".

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Problème n°2 :

- Les entreprises se livrent une "**guerre de la mise à jour**".
- Il faut que l'entreprise soit capable de faire évoluer son application de façon très fréquente et de répondre rapidement aux nouvelles fonctionnalités que propose la concurrence.
- Il faut donc passer à des architectures qui permettent de faire des mises à jour qui visent des **composants ciblés** , très rapidement, de façon fiable et sans se soucier des éventuelles conséquences sur le reste de l'application.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Problème n°3

- Les technologies utilisées pour développer ces applications **évoluent très vite** et les nouveautés offrent parfois des avantages énormes.
- Comment les entreprises peuvent-elles s'adapter rapidement pour tirer profit de ces évolutions ?
- L'entreprise doit adopter une architecture qui lui permettra de **passer d'une technologie à l'autre sur des portions individuelles** de son application très facilement.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Naissance des MSA

- Il faut donc que les entreprises conçoivent leurs applications dès le départ à partir d'architectures répondant à ces besoins et qui sont parfaitement adaptées au **cloud**.
- C'est ainsi que l'architecture **Microservices** est apparue, apportant une réponse concrète à toutes ces préoccupations et la promesse d'obtenir, au bout, des applications dites "**Cloud-native**".

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Que sont les microservices?

- Les microservices sont un modèle d'architecture **orienté service** dans lequel les applications sont construites comme un ensemble de diverses unités de service indépendantes les plus petites.
- Il s'agit d'une approche d'ingénierie logicielle qui se concentre sur la décomposition d'une application en modules à **fonction unique** avec des interfaces bien définies et **faiblement couplés**.
- Ces modules peuvent être déployés et exploités indépendamment par de petites équipes qui possèdent l'intégralité du cycle de vie du service.

Elies Jebri

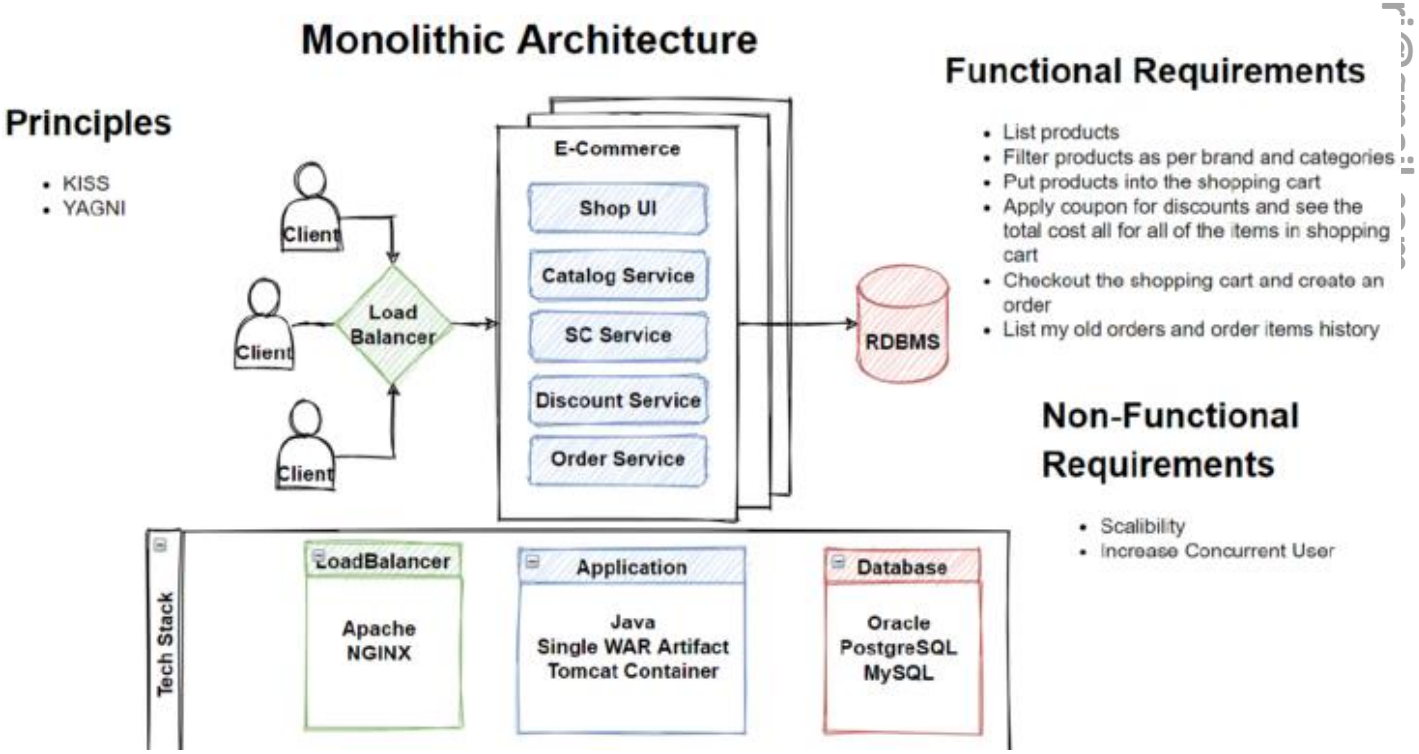
Elies Jebri : elies.jebri@gmail.com

Monolithic vs. SOA vs. Microservices

Elies Jebri : elies.jebri@gmail.com

Exemple e-commerce

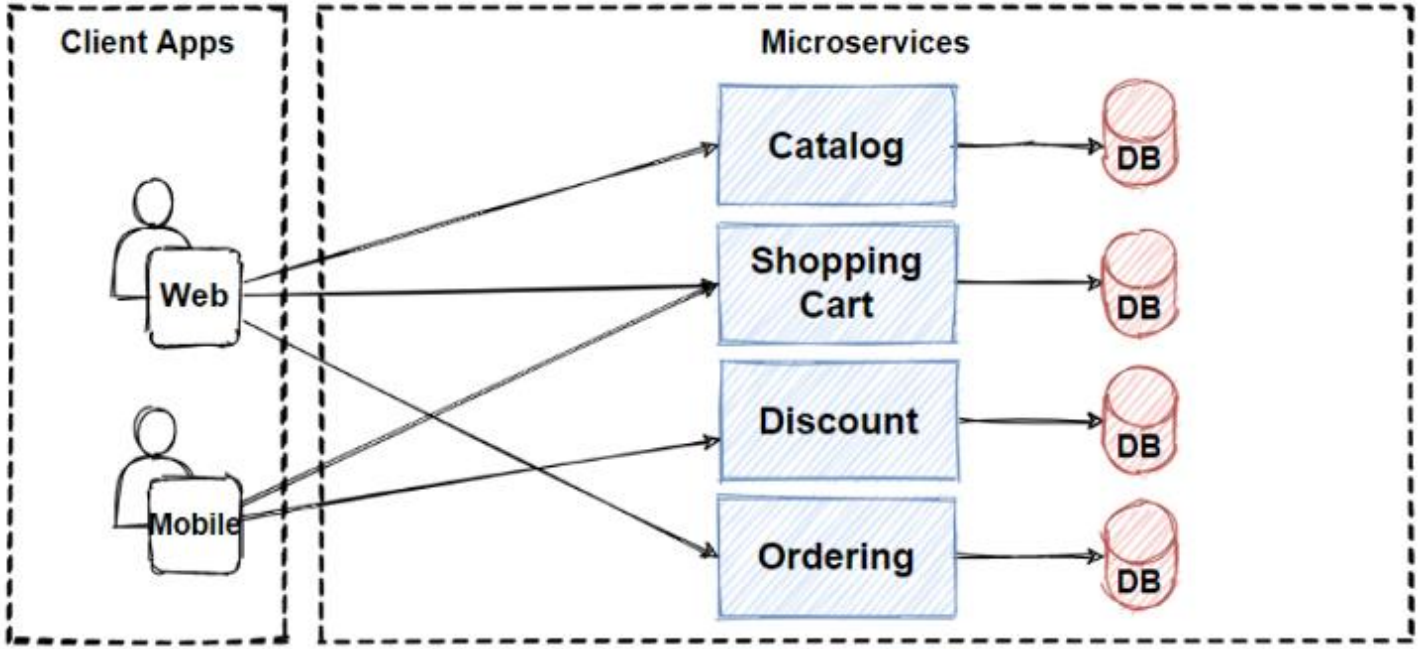
- Dans toute application de commerce électronique, il existe des fonctionnalités standard telles que la recherche, évaluation & notation et paiement.
- Ces fonctionnalités sont accessibles aux clients via leur navigateur ou leurs applications.
- Le code des différentes fonctionnalités se trouve sur le même serveur (monolythe) ou plusieurs (SOA).
- Pour faire évoluer l'application, vous devez exécuter plusieurs instances (serveurs) de ces applications.



Elies Jebri : elies.jebri@gmail.com

E-commerce microservice

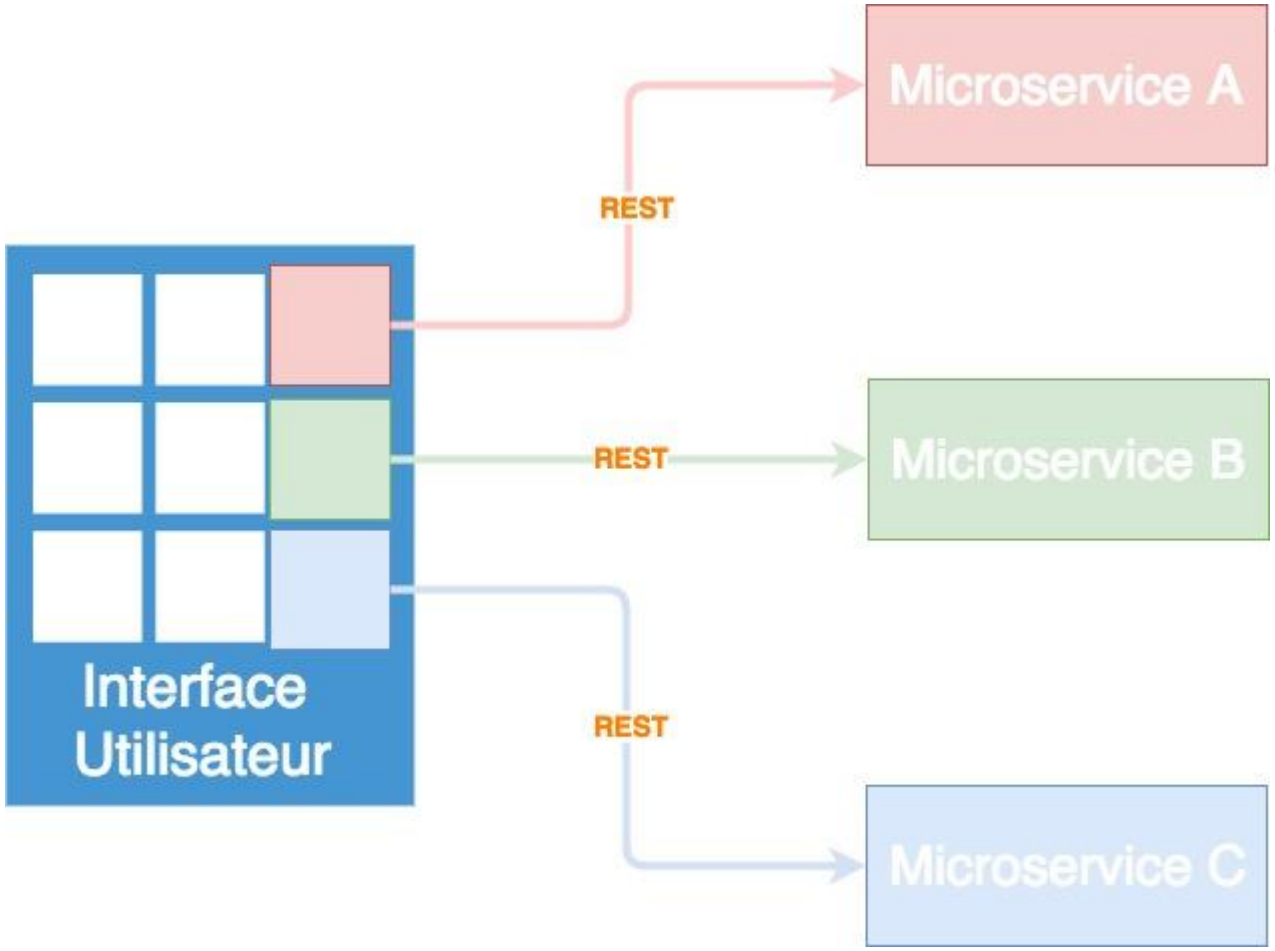
- Chaque microservice est axé sur une capacité commerciale unique.
- Catalogue, Panier ont chacun leur instance (serveur) et communiquent entre eux.
- La communication entre les microservices est une communication sans état où chaque paire de requête et de réponse est indépendante.
- Dans l'architecture Microservice, les données sont fédérées. Chaque microservice a son magasin de données distinct.



Elies Jebri : elies.jebri@gmail.com

Principe de l'architecture Microservices

- Découper une application en **petits services**, appelés Microservices, **parfaitement autonomes** qui exposent une **API (REST, …)** que les autres Microservices pourront consommer.



Elies Jebri : elies.jebri@gmail.com

LES API REST

Elies Jebri : elies.jebri@gmail.com

Qu'est-ce qu'une API ?

- API signifie Application Programming Interface.
- Les sites web et les applications ont besoin de communiquer et échanger des données.
- Une API est une interface pour les applications, un framework

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

REST

- REST signifie “Representational State Transfer”
- Style d’architecture
- Les API REST sont basées sur HTTP
- L’échange est basé sur des requêtes client et serveur
- Ce sont des méthodes qui définissent les requêtes que le client peut effectuer, dont GET, PUT, POST, PATCH, DELETE

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Les critères REST

- Les API REST imitent la façon dont le web lui-même marche dans les échanges entre un client et un serveur. Une API REST est :
  - Sans état
  - Cacheable (avec cache = mémoire)
  - Orientée client-serveur
  - Avec une interface uniforme
  - Avec un système de couche
  - Un code à la demande (optionnel)

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

JSON

- Le format de données JSON (JavaScript Object Notation) constitue le standard actuel pour les échanges de données sur le Web, notamment avec AJAX.
- Il s'agit d'une syntaxe pour décrire des informations structurées sous une forme proche des objets JavaScript.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Voici un exemple de document JSON qui décrit deux voitures

```
{
  "voitures" : [
    { "modèle" : "Peugeot",
      "couleur" : "bleu",
      "immatriculation" : 2008,
      "révisions" : [ 2012, 2014 ]
    },
    { "modèle" : "Citroën",
      "couleur" : "blanc",
      "immatriculation" : 1999,
      "révisions" : [ 2003, 2005, 2007, 2009, 2011, 2013 ]
    }
  ]
}
```

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

CRUD Create Read Update Delete

- CRUD (pour create, read, update, delete) (parfois appelé SCRUD avec un "S" pour search) désigne les quatre opérations de base pour la persistance des données, en particulier le stockage d'informations en base de données.
- Soit :
  - create : créer
  - read : lire
  - update : mettre à jour
  - delete : supprimer
- Plus généralement, il désigne les opérations permettant la gestion d'une collection d'éléments.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

CRUD Utilisation dans les interfaces logicielles

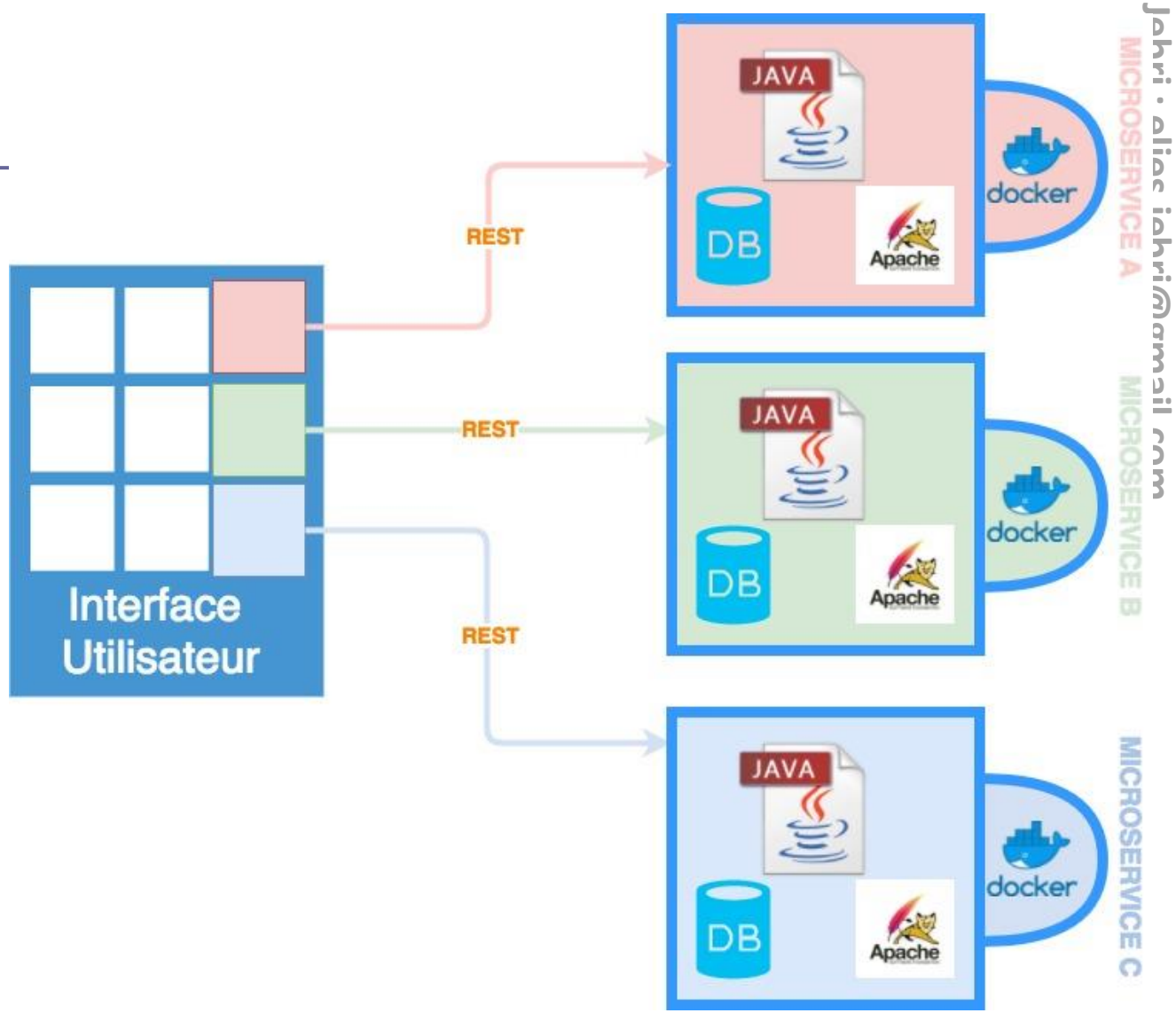
- Par exemple, une application carnet d'adresse dont l'élément le plus simple est un contact, doit permettre à l'utilisateur de :
  - Créer des contacts
  - Lire un contact (liste, zone de recherche…)
  - Mettre à jour un contact
  - Supprimer un contact
- Si l'une ou l'autre de ces opérations fondamentales manque, le logiciel sera considéré comme incomplet.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Principe

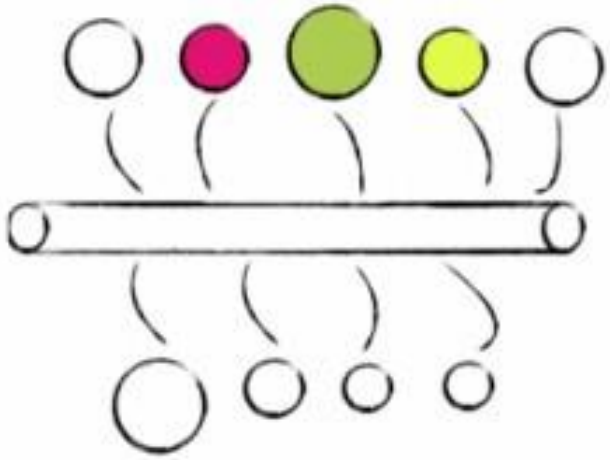
- Chaque Microservice est parfaitement autonome : il a sa propre base de données, son propre serveur d'application (tomcat, jetty, etc.), ses propres librairies et ainsi de suite.
- La plupart du temps ces Microservices sont chacun dans un container **Docker**, ils sont donc totalement indépendants y compris vis-à-vis de la machine sur laquelle ils tournent.



Elies Jebri · elies.jebri@gmail.com

Microservice : Concepts

- Des concepts présentés dans les slides précédents restent valables (réutilisabilité, autonomie, sans état…)
- Huit nouveaux aspects caractérisent un microservice
  - Fonctionnalité unique
  - Flexibilité technologique
  - Equipe réduite <=10
  - Déploiement ciblé
  - Montée en charge « scalabilité »
  - Tests facilités



Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Une seule fonctionnalité

- Un microservice doit réaliser une seule fonctionnalité de l'application globale
- Un microservice peut contenir toutes les couches logicielles (IHM, middleware et base de données)
- Un microservice possède un contexte d'exécution séparé des autres (exemple : machine virtuelle ou conteneur)

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Flexibilité technologique

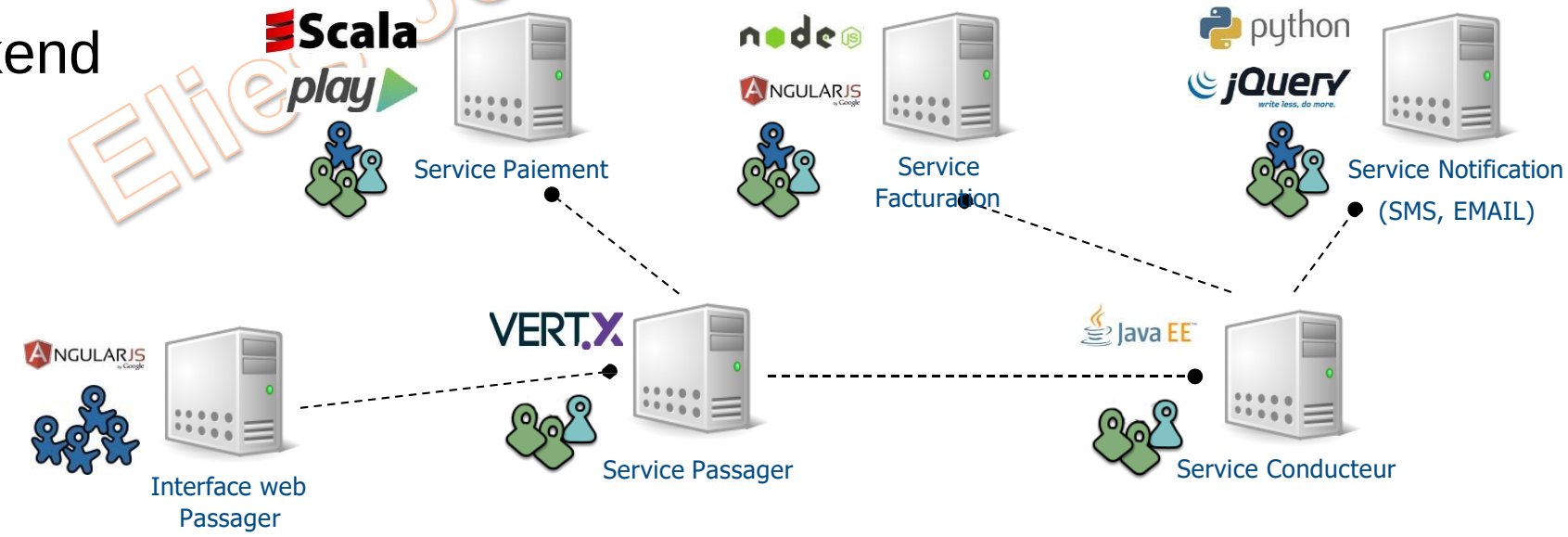
- Utiliser les bons langages et frameworks selon la fonction-nalité à réaliser
  - Play/Scala
  - Java EE (JSF/EJB/Java), Spring
  - JQuery/AngularJS/NodeJS
  - Django/Python
  - Symfony/PHP

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Equipe réduite – two pizza team

- Chaque microservice à sa propre équipe de développement
- L'équipe de développement orientée fonctionnalité est réduite et pluridisciplinaire
  - Développeurs backend
  - Développeurs web
  - Administrateurs bases de données



Elies Jebri
Elies.jebri.elies@gmail.com

Déploiement ciblé

- Evolution d'une certaine partie sans tout redéployer
- Un seul livrable à partir d'un seul code source
- Moins de coordination entre équipes quand il y a un seul déploiement
  - Plus souvent
  - Moins de risque
  - Plus rapide

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Montée en charge / scalabilité

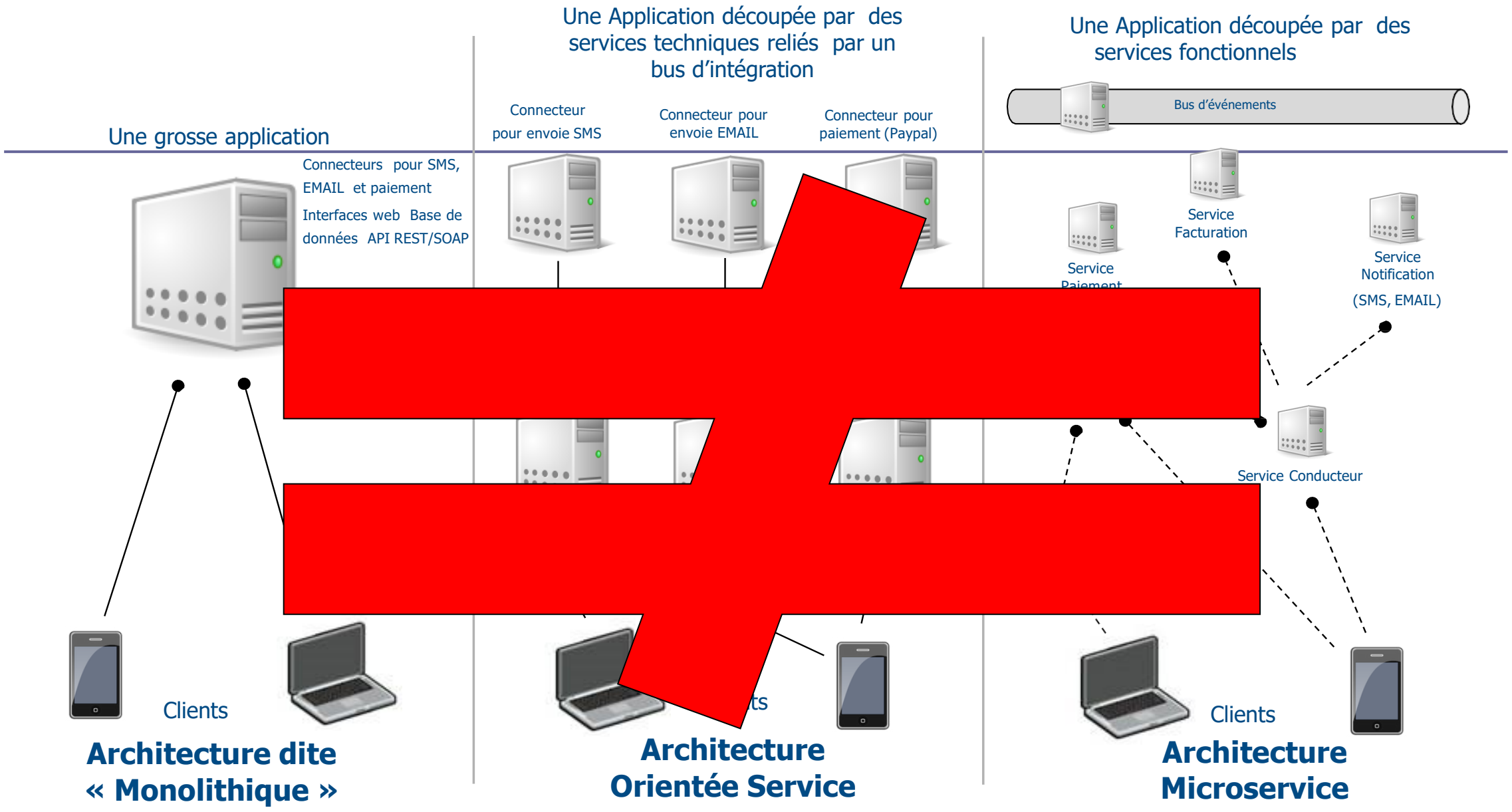
- Forte sollicitation sur un microservice ?
  - page web : Amazon en période de «Black Friday»
  - batch : compression de vidéos
- Comme chaque fonctionnalité est isolée possibilité de multiplier le nombre d’instances d’un microservice.

Facilite les tests

- Comme chaque fonctionnalité est isolée dans un microservice il est plus facile de les tester,
- Les tests unitaires concernent la partie intrinsèque d'un microservice => on sait déjà faire
- Quels types de tests restants ?
  - Service : stresser l'API exposée du microservice
  - Bout-en-bout : intégrer les autres microservice
- Si c'est facile à tester alors c'est facile à déployer (principe du déploiement continu)

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

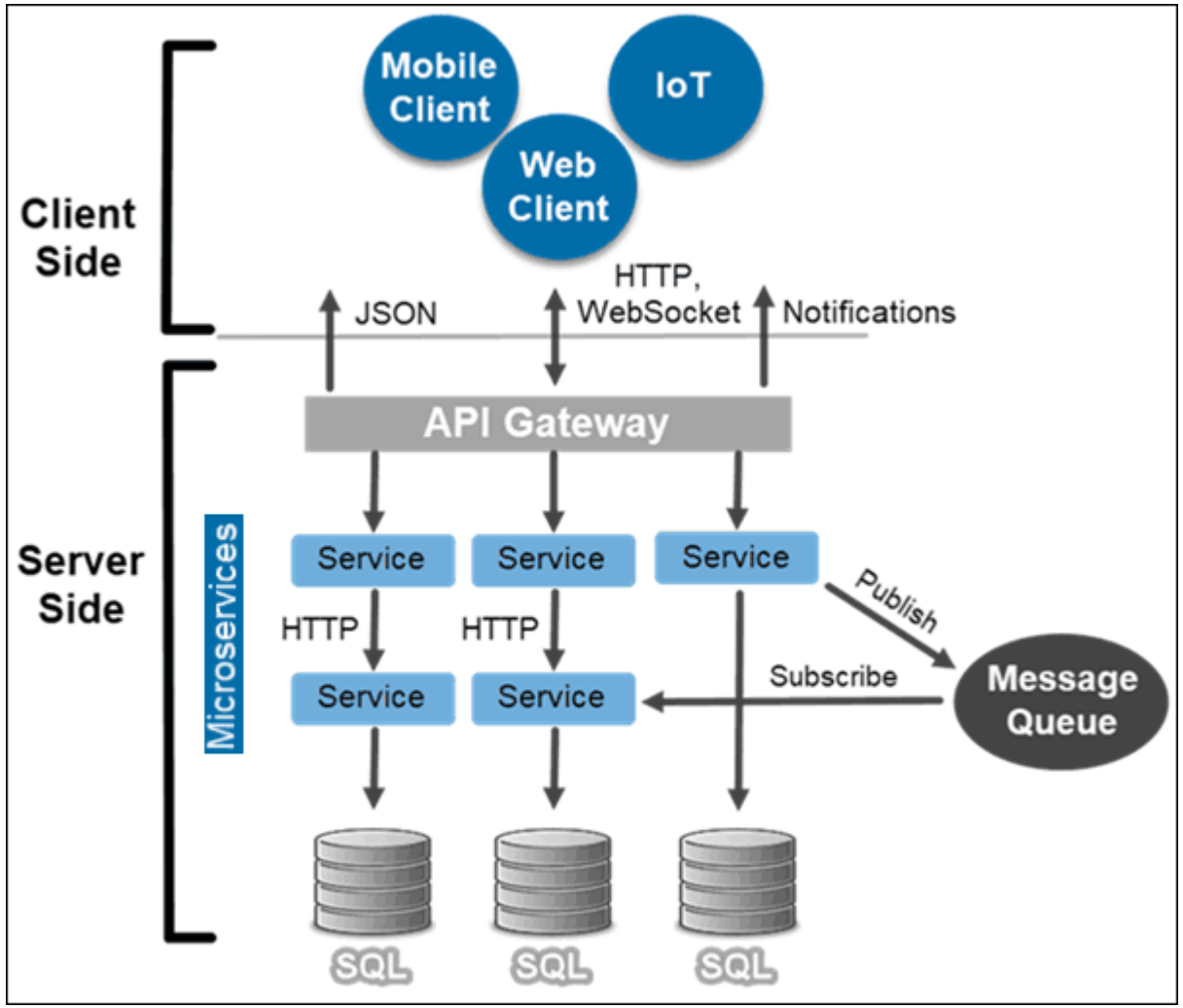


Elies Jebri : elies.jebri@gmail.com

FAIRE ÉVOLUER L'ARCHITECTURE

Elies Jebri : elies.jebri@gmail.com

Architecture microservice



- Une architecture microservice est donc un ensemble de microservices autonomes
- Pas de bus d'intégration (ESB) car il y a de la logique et donc cela force le couplage
- Utilisation d'un Bus d'événements pour le faible couplage
- Utilisation d'un Load Balancer pour équilibrer la charge

Elies Jebri : elies.jebri@gmail.com

Développer des architectures microservice

- Savoir Isoler un microservice
- Savoir Coder le contenu du microservice
- Savoir faire Communiquer des microservices
- Savoir Composer les microservices
- Savoir Répartir les charges

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Communication Microservices

- L'un des plus grands défis lors du passage à une application basée sur des microservices est de changer le mécanisme de communication.
- Parce que les microservices sont distribués et les microservices communiquent entre eux par communication inter-services au niveau du réseau.
- Par conséquent, les services doivent interagir à l'aide de protocoles de communication interservices tels que HTTP, gRPC ou le protocole AMQP des courtiers de messages (Message Broker).

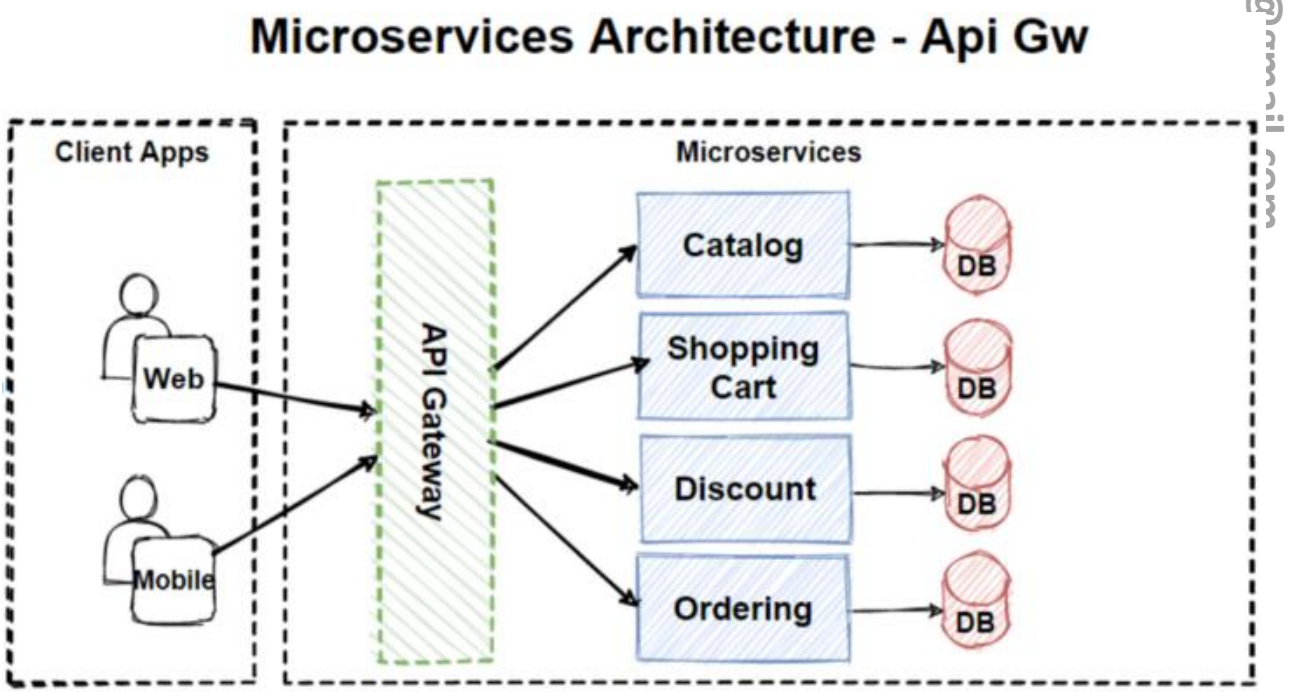
- Étant donné que les microservices sont une structure complexe en services développés et déployés indépendamment, nous devons être prudents lorsque nous considérons les types de communication et les gérer dans les phases de conception.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

API Gateway

- Nous avons un nouveau panel d'outils à notre disposition comme l'API Gateway.
- Permet de définir des **endpoints** sécurisés accessibles de l'extérieur et de rediriger l'appel vers un service de traitement (génération d'un événement, lancement d'une fonction) qui une fois fini pourra retourner des données selon les besoins.



Elies Jebri : elies.jebri@gmail.com

API Gateway

- Collecter la demande du client dans un point d'entrée unique et acheminer la demande vers les microservices internes.

- Traiter les demandes des clients et les acheminer vers les microservices internes, (agréger) également plusieurs microservices internes en une seule demande client.

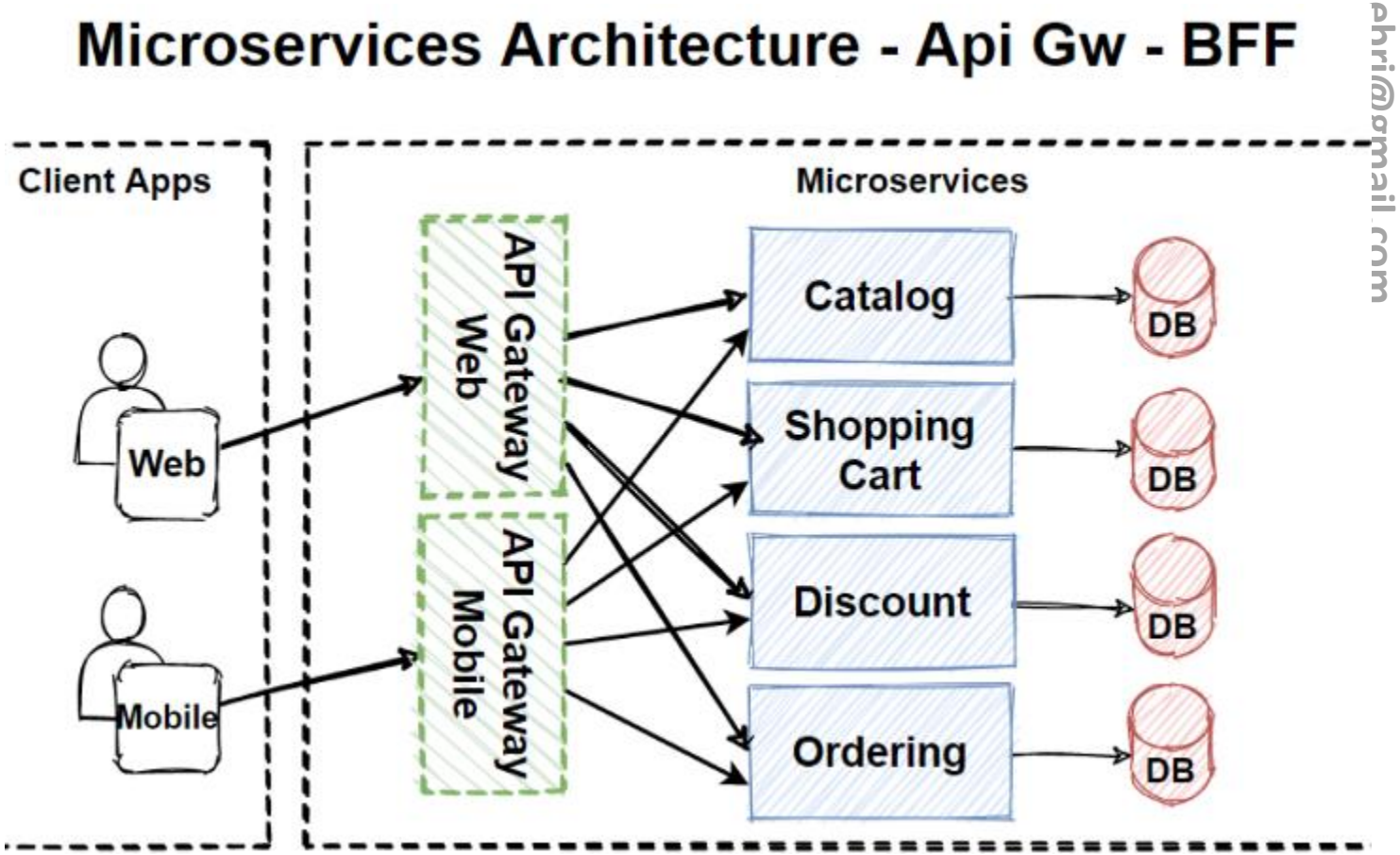
- Effectuer des tâches transversales telles que l'authentification et l'autorisation, la limitation du débit, etc.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Design Backends for Frontends pattern BFF

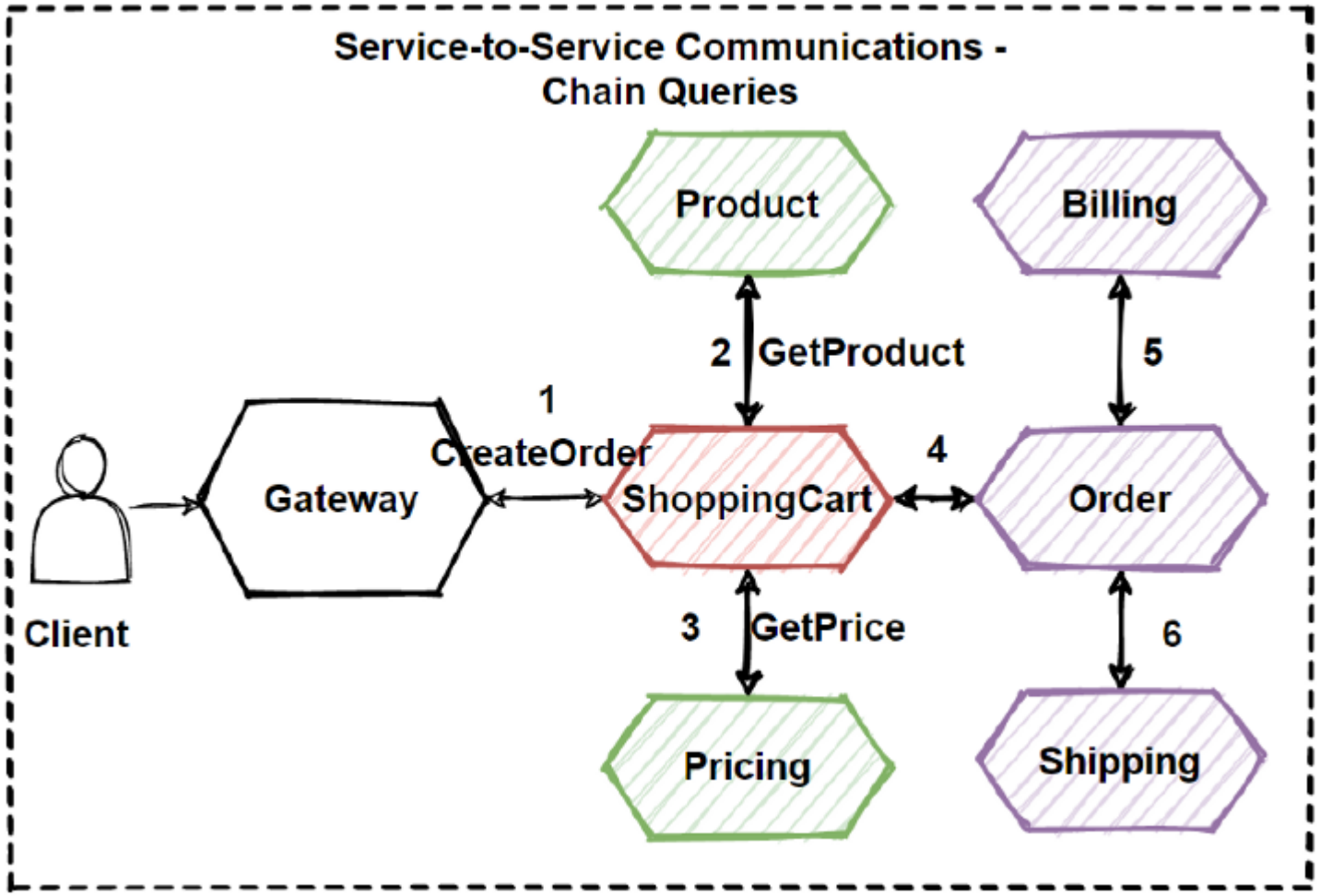
- Plusieurs passerelles API pour répondre au mieux aux besoins de l'environnement frontal, sans se soucier d'affecter les autres applications frontales.
- Le modèle Backend for Frontends fournit une orientation pour la mise en œuvre de plusieurs passerelles.



Elies Jebri : elies.jebri@gmail.com

Communications entre les microservices internes dorsaux

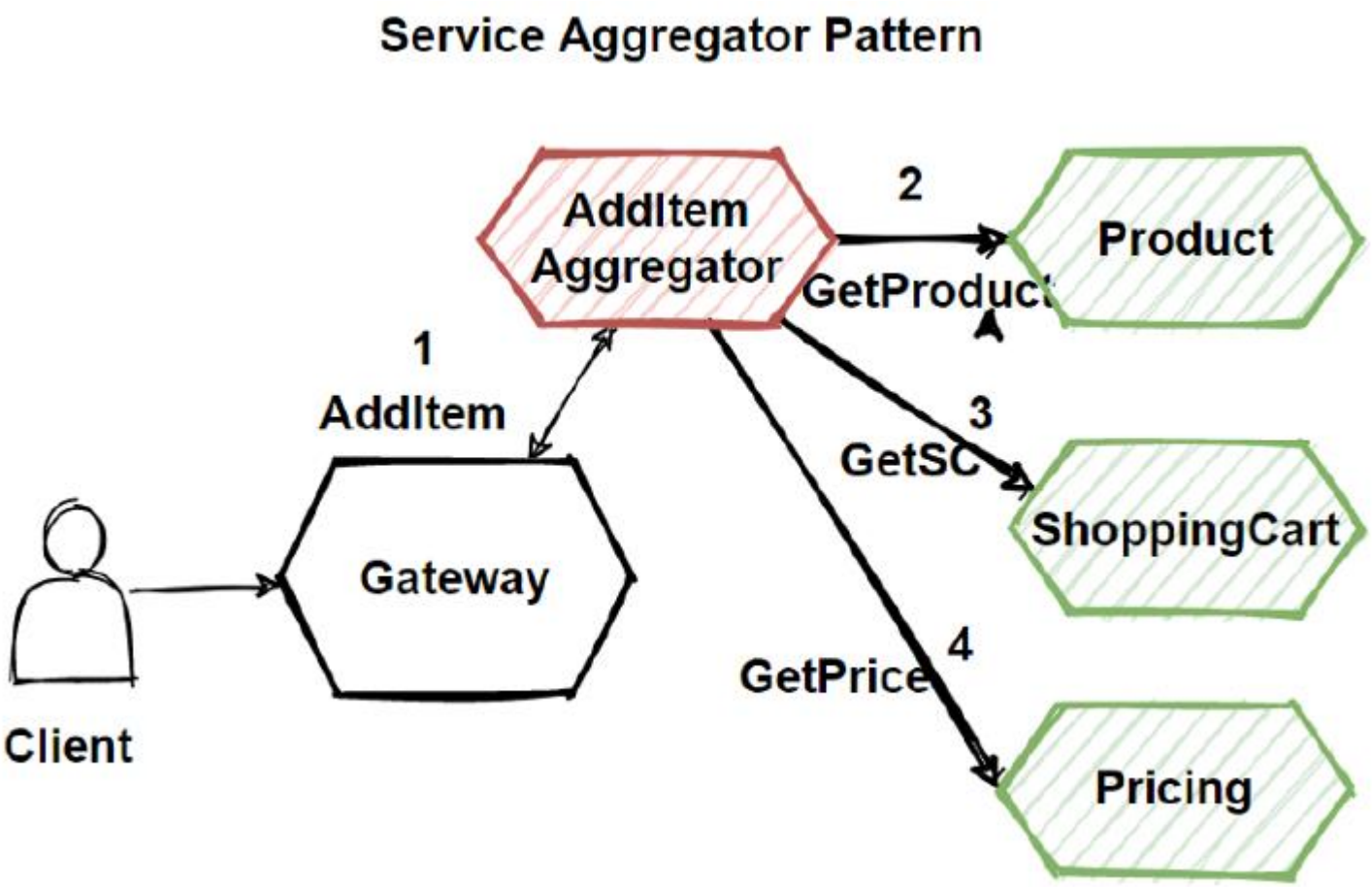
- Que se passe-t-il si les demandes du client doivent visiter plusieurs microservices internes ?
- Comment gérer les communications internes des microservices ?



Elies Jebri : eli

Modèle d'agrégateur de services

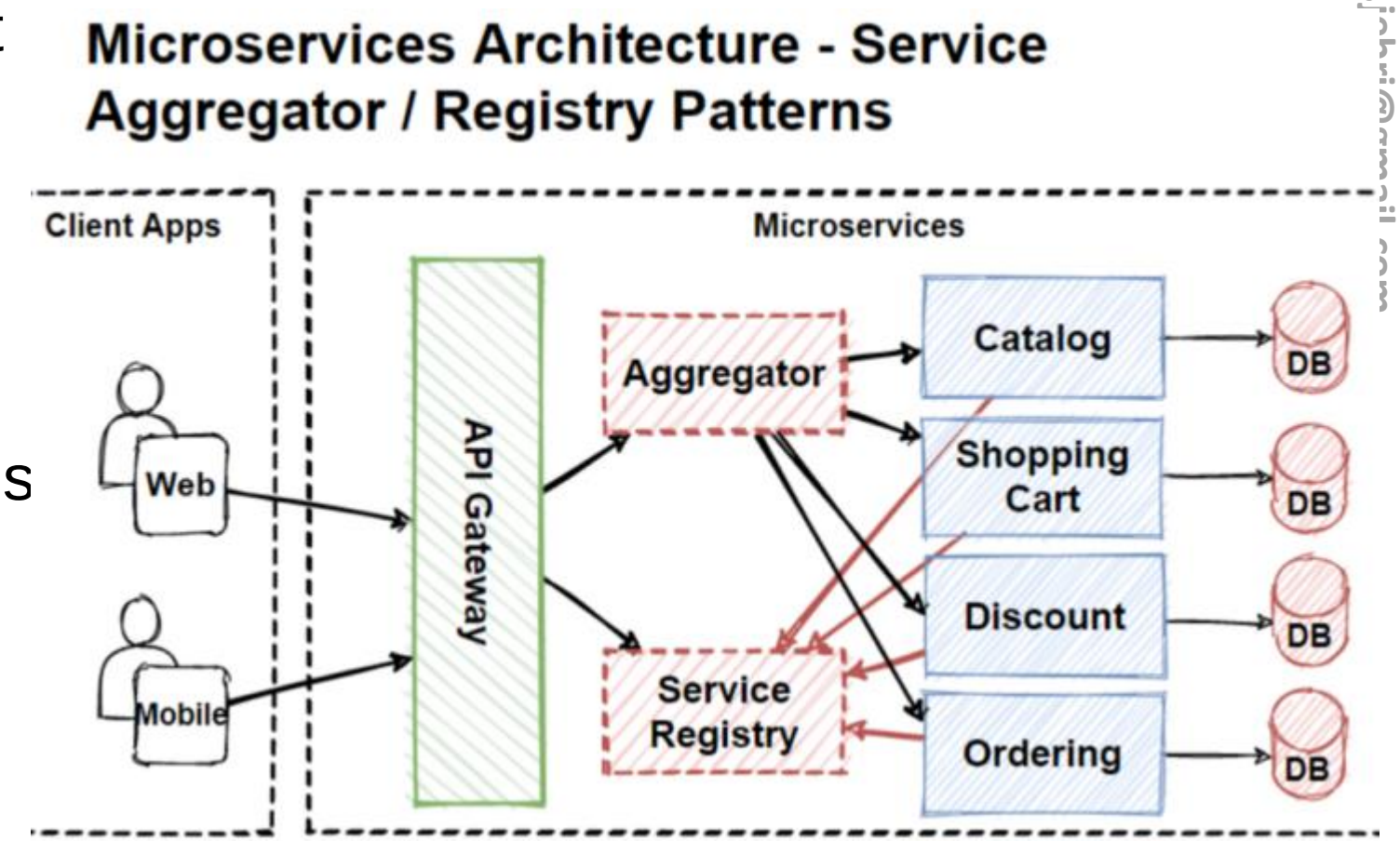
- Permet de minimiser les communications de service à service.
- L'agrégateur de services reçoit une demande du client ou de l'api gw, puis envoie les demandes de plusieurs microservices principaux internes, puis combine les résultats et répond à la demande d'origine dans une structure de réponse.



Elies Jebri : elies.jebri@gmail.com

Modèle de registre de services

- Un registre de services est une base de données qui sert au stockage des structures de données utilisées pour la communication entre applications.
- C'est un point central où les développeurs peuvent enregistrer et trouver des schémas utiles pour créer des applications spécifiques.



Elies Jebri : elies.jebri@gmail.com

Communiquer

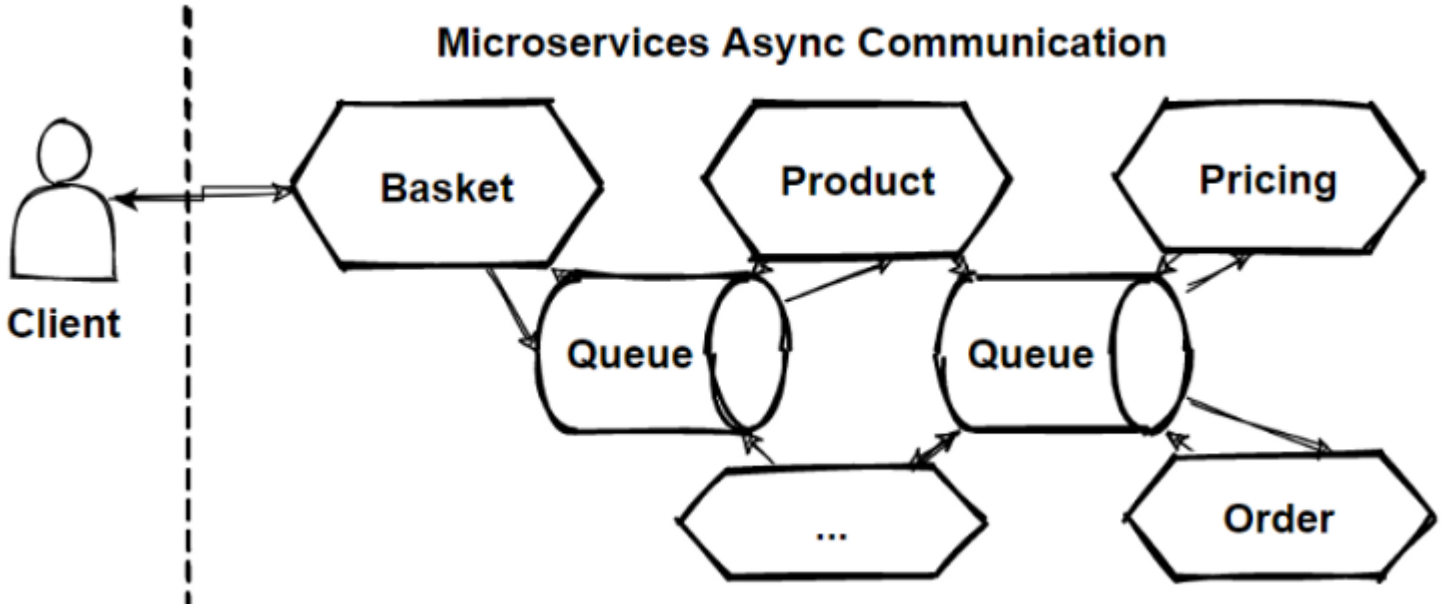
- La communication entre des microservices peut se faire
- « conjointement ou pas » via
  - des services web (REST ou SOAP) => communication synchrone
  - un bus d'événements => communication asynchrone
- Bus d'événements basés sur le protocole AMQP (Advanced Message Queuing Protocol)
- Solutions de bus d'événements basé sur AMQP
  - RabbitMQ (Pivotal) => le plus utilisé et documenté
  - ActiveMQ, Apollo (Apache)
  - Qpid (Apache)
  - HornetQ (Jboss)

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Communication asynchrone basée sur les messages des microservices

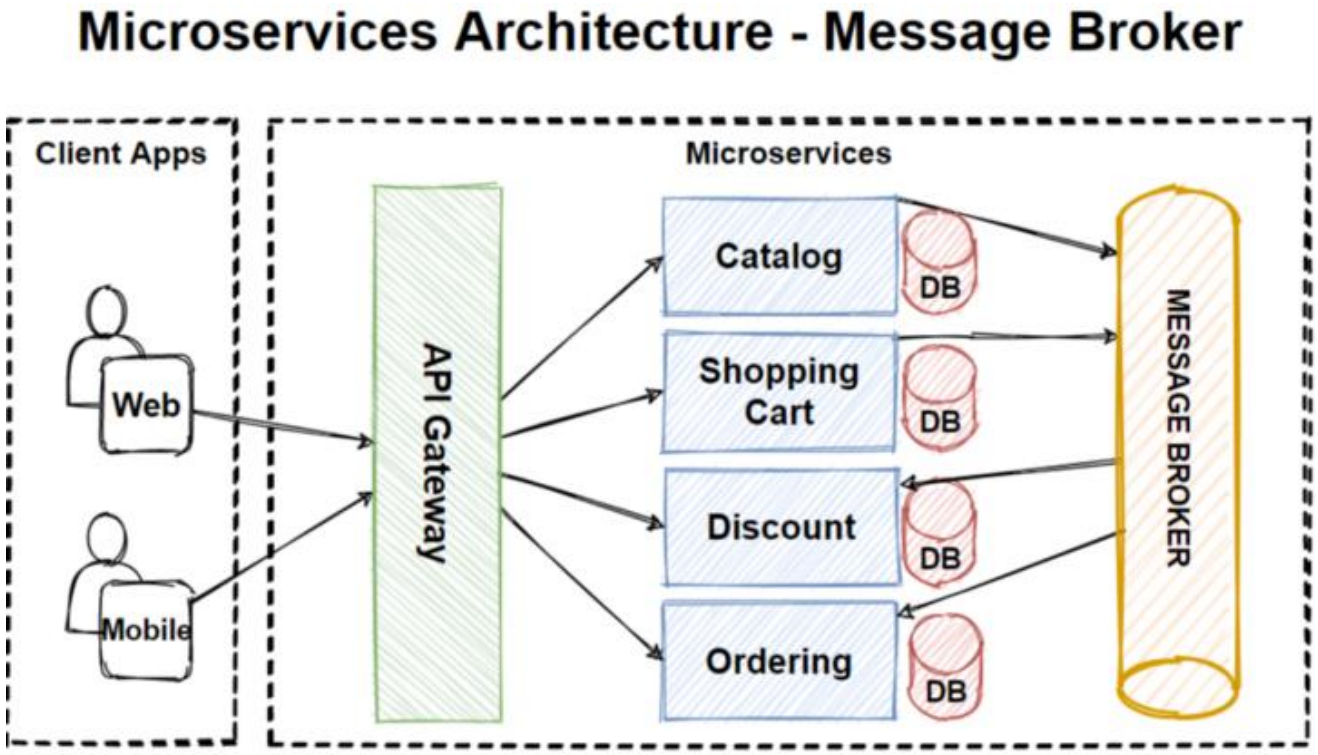
- Lorsqu'il s'agit de plusieurs microservices qui doivent s'appeler et attendre de longues opérations jusqu'à ce qu'ils soient terminés, nous devons utiliser la communication asynchrone.



Elies Jebri : elies.jebri@gmail.com

Broker de messages Pub/Sub

- Ajout d'un Pub/Sub Message Broker pour fournir aux microservices des composants de communication asynchrone.

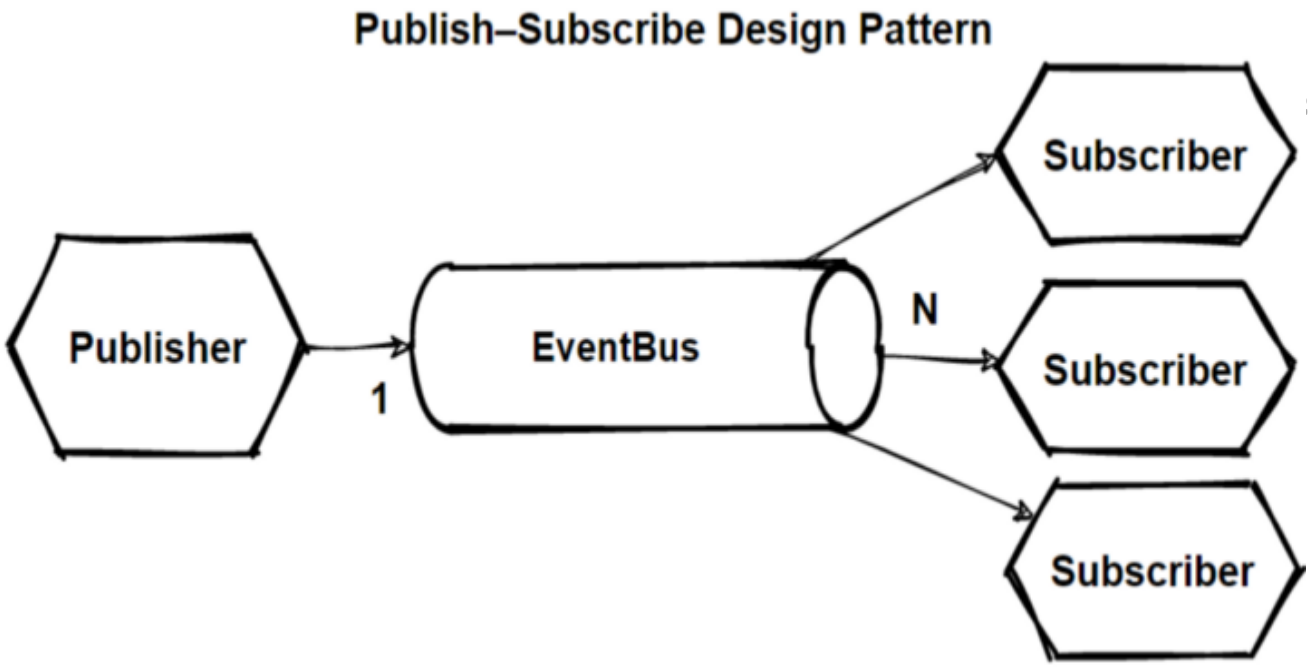


Elies Jebri : elies.jebri@gmail.com

Modèle de conception de publication–abonnement

- Les éditeurs n'envoient pas les messages directement aux abonnés.
- catégoriser les messages publiés et les envoyer aux systèmes de courtier de messages sans savoir quels abonnés sont présents.

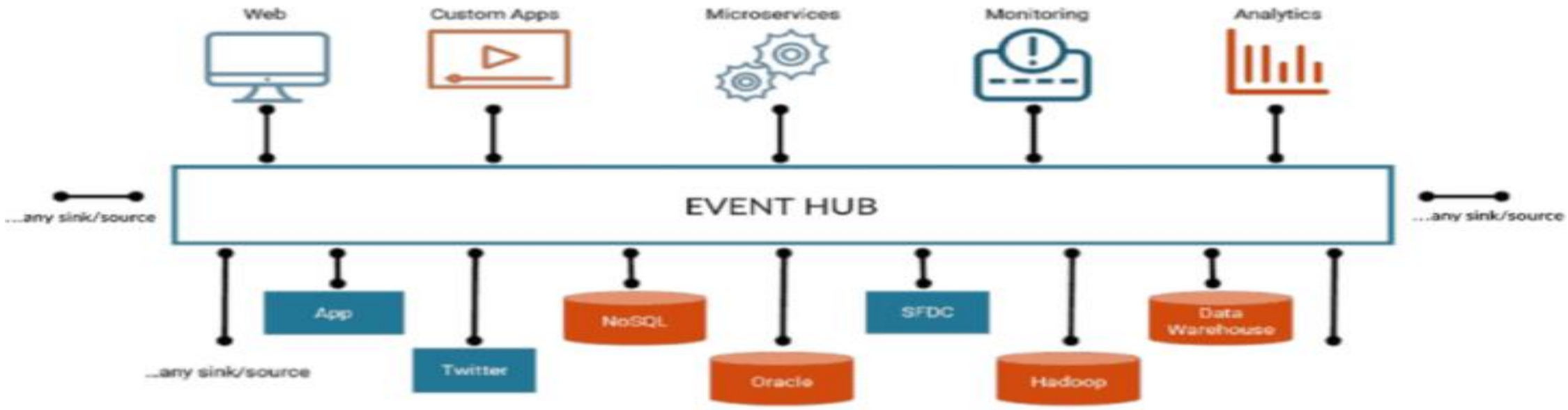
- De même, les abonnés expriment leur intérêt et ne reçoivent que les messages qui les intéressent, sans savoir quels éditeurs leur envoient.



Elies Jebri : elies.jebri@

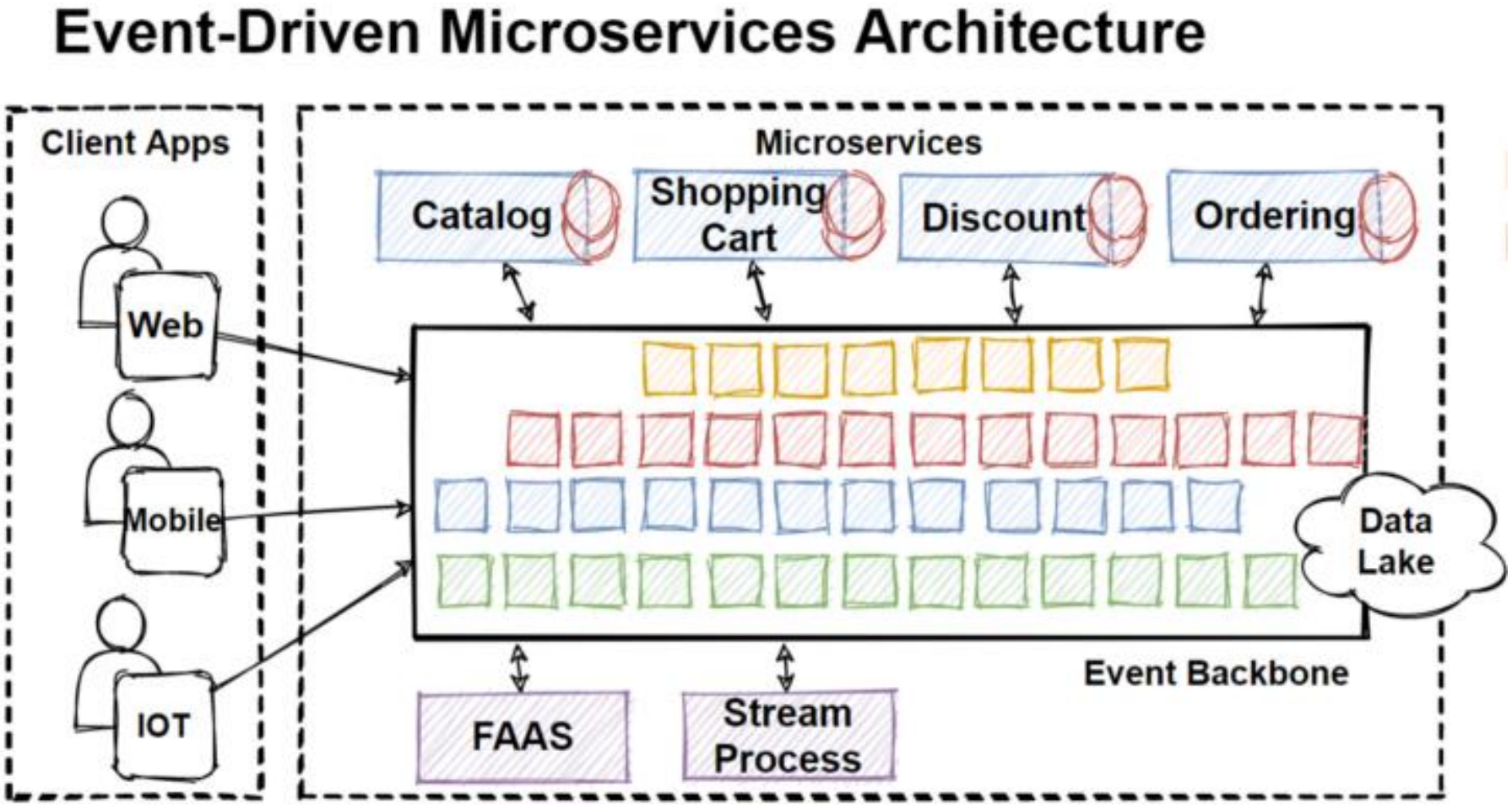
Architecture de microservices pilotée par les événements

- L'architecture microservices pilotée par les événements consiste à communiquer avec les microservices via des messages d'événement.
- Beaucoup d'innovations: plates-formes de messagerie en temps réel, le traitement de flux, les hubs d'événements, le traitement en temps réel, le traitement par lots, l'intelligence des données, etc.



Elies Jebri : elies.jebri@gmail.com

Notre application e-commerce avec l'architecture Event-Driven Microservices



Elies Jebri : elies.jebri@gmail.com

Composer

- La composition va permettre de résoudre le problème d'orchestration des conteneurs
- Exemple simple pour comprendre cette problématique
  - A = un conteneur pour la base de données (MySQL)
  - B = un conteneur applicatif
  - A doit être démarré avant B et B a besoin d'accéder à A (ouverture de port)
- Solutions du marché
  - Docker Compose (Docker inc.) => le plus simple
  - Docker Swarm (Docker inc.)
  - Kubernetes (Google)
  - Mesos (Apache)
  - Fleet (CoreOS)
  - Crane (https://github.com/michaelsauter/crane)
  - Consul

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Répartir la charge : reverse proxy / LB

- Problématique du pourquoi utiliser un reverse proxy
  - Plusieurs conteneurs Docker vont être créés
  - Plusieurs conteneurs d'une même image peuvent être créées
- Reverse Proxy ou « Proxy inversé »
  - Donne accès depuis un réseau externe aux conteurs d'un réseau interne
  - Distribuer en fonction de la charge les requêtes web aux conteneurs les moins occupés (d'une même image par exemple)
- Solutions libres du marché
  - Nginx (Nginx inc.) => largement utilisé et très simple
  - Haproxy
  - Hipache (Docker inc.)
  - Træfik

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Avantages des microservices

- Chaque unité de l'ensemble de l'application doit être la plus petite et doit pouvoir atteindre un objectif métier spécifique.
- Le démarrage du service est relativement rapide
- L'isolement des pannes est facile. Même si un service tombe en panne, un autre peut continuer à fonctionner.
- Tous les microservices doivent être faiblement couplés afin que les modifications apportées à l'un n'affectent pas l'autre.
- Les entreprises peuvent déployer plus de ressources vers des services générant un retour sur investissement plus élevé

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Avantages des microservices

- Davantage de ressources matérielles pourraient être allouées au service fréquemment utilisé.
- Les microservices restent toujours cohérents et disponibles en permanence.
- Les données sont fédérées. Cela permet à un microservice individuel d'adopter un modèle de données le mieux adapté à ses besoins.
- Petites équipes ciblées. Développement parallèle et plus rapide
- La modification du modèle de données d'un microservice n'affecte pas les autres microservices.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Avantages des microservices

- Interagit avec d'autres microservices en utilisant des interfaces bien définies
- Les microservices fonctionnent sur le principe qui se concentre sur les produits, pas sur les projets
- Pas de dépendances croisées entre les bases de code. Vous pouvez utiliser différentes technologies pour différents microservices.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Défis des microservices

- Les MicroServices dépendent les uns des autres et ils devront communiquer entre eux.
- Par rapport aux systèmes monolithiques, il existe davantage de services à surveiller qui sont développés à l'aide de différents langages de programmation .
- Comme il s'agit d'un système distribué, il s'agit d'un modèle intrinsèquement complexe.
- Différents services auront leur mécanisme séparé, ce qui entraînera une grande quantité de mémoire pour des données non structurées.
- Gestion efficace et travail d'équipe requis pour éviter les problèmes en cascade
- Reproduire un problème sera une tâche difficile lorsqu'il est parti dans une version et revient dans la dernière version.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Défis des microservices

- Le déploiement indépendant est compliqué avec les microservices.
- L'architecture des microservices entraîne de nombreux frais généraux d'opérations.
- Il est difficile de gérer l'application lorsque de nouveaux services sont ajoutés au système
- Un large éventail de professionnels qualifiés est nécessaire pour prendre en charge les microservices distribués de manière hétérogène

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Défis des microservices

- La pile technologique des microservices pourrait être très volumineuse
- Par nature les applications microservice sont Full Stack
- La logique métier ne réside plus dans l'application

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Bilan

- Le ‘Time to market’
  - Création de valeur
  - Réactivité aux évolutions du marché
  - Satisfaction des utilisateurs
- L’agilité technologique
- Modernisation facilitée
- Évolutivité
- Fiabilité

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Bilan

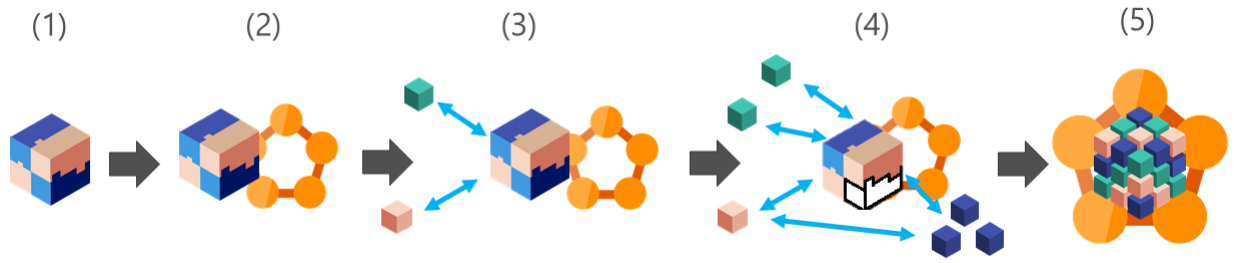
- L’architecture microservices n’est pas la solution à tous les problèmes de performances d’un applicatif
- Problèmes courants
  - Modélisation
  - Persistance des données
  - Transaction
  - Front-end non adapté
- Les microservices adressent le problème d’architecture et sont là pour faciliter la montée en charge
- Attention : il n’a jamais été précisé que les architectures monolithiques étaient mortes

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Pour aller plus loin

- Les bonnes pratiques pour passer à une architecture microservices => à partir d'une application monolithique
- Déploiement vers des clusters
- Monitoring
- Transaction
- Sécurité



Elies Jebri : elies.jebri@gmail.com

THE TWELVE-FACTORS APP

Elies Jebri : elies.jr

Introduction

12 factor Application est une méthodologie pour concevoir des logiciels en tant que service qui :

- Utilisent des formats déclaratifs pour mettre en œuvre l'automatisation, pour minimiser le temps et les coûts pour que de nouveaux développeurs rejoignent le projet;
- Ont un contrat propre avec le système d'exploitation sous-jacent, offrant une portabilité maximum entre les environnements d'exécution;
- Sont adaptés à des déploiements sur des plateformes cloud modernes, rendant inutile le besoin de serveurs et de l'administration de systèmes;
- Minimisent la divergence entre le développement et la production, ce qui permet le déploiement continu pour une agilité maximum;
- et peuvent grossir verticalement sans changement significatif dans les outils, l'architecture ou les pratiques de développement;

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

12 Factors

Codebase
One codebase tracked in revision control & many deploys

Logging
Treat logs as streams

Dev/Prod Parity
Keep dev, prod & staging as close as possible.

Backing Services
Treat backing service as attached resource

Config
Store config in the environment

Disposability
Maximize the robustness with fast boot and fast shutdown

Backing Services
Treat backing service as attached resource

Dependencies
Explicit declare and isolate dependencies.

Concurrency
Scale out via the process model

Admin Processes
Run admin/management task as one off processes

Build ,Release, Run
Strictly separate build and run stages.

Port Binding
Export services via port binding

Elies Jebri : elies.jebri@gmail.com

Les 12 facteurs

I. Base de code

- Une base de code suivie avec un système de contrôle de version, plusieurs déploiements

II. Dépendances

- Déclarez explicitement et isolez les dépendances

III. Configuration

- Stockez la configuration dans l'environnement

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Les 12 facteurs

IV. Services externes

- Traitez les services externes comme des ressources attachées

V. Assemblez, publiez, exécutez

- Séparez strictement les étapes d'assemblage et d'exécution

VI. Processus

- Exécutez l'application comme un ou plusieurs processus sans état

VII. Associations de ports

- Exportez les services via des associations de ports

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Les 12 facteurs

VIII. Concurrence

- Grossissez à l'aide du modèle de processus

IX. Jetable

- Maximisez la robustesse avec des démarrages rapides et des arrêts gracieux

X. Parité dev/prod

- Gardez le développement, la validation et la production aussi proches que possible

XI. Logs

- Traitez les logs comme des flux d'évènements

XII. Processus d'administration

- Lancez les processus d'administration et de maintenance comme des one-off-processes

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

SoC Separation of Concerns

LA SÉPARATION DES PRÉOCCUPATIONS

Elies Jebri : elies.jebri@gmail.com

Separation Of Concern SoC, c'est quoi?

- En développement logiciel, la séparation des responsabilités vise à découper un programme en un ensemble de sous-programmes où chaque sous-programme a la responsabilité de traiter un aspect du programme.
- Il en est de même pour chaque sous-programme qui est divisé en composants et ainsi de suite, de telle sorte que chaque section aborde une préoccupation distincte.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

SoC, c’est quoi?

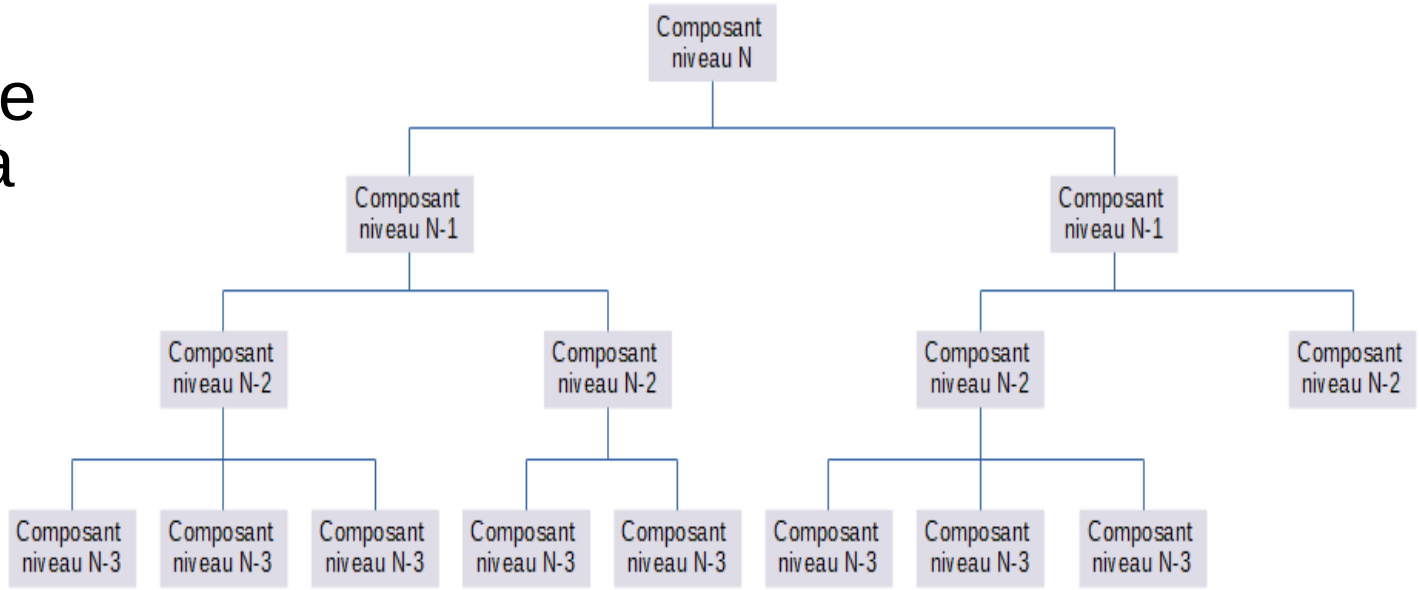
- Une préoccupation est un ensemble d'informations qui affecte le code d'un programme informatique.
- Une préoccupation peut être aussi générale que "les détails du matériel d'une application", ou aussi spécifique que "le nom de la classe à instancier".
- Un programme qui incarne bien SoC est appelé programme modulaire.
- La modularité, est obtenue en encapsulant des informations dans une section de code qui a une interface bien définie.
- L'encapsulation est un moyen de cacher l'information.
- Les conceptions en couches dans les systèmes d'information sont un autre mode de réalisation de la séparation des préoccupations (par exemple, couche de présentation, couche de logique métier, couche d'accès aux données, couche de persistance).

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Découpage

- La vraie difficulté réside dans la façon de découper.
- Il faut attribuer à chaque composant d'un niveau une **responsabilité claire**, et à chaque composant de niveau inférieur, il faut donner un morceau de la responsabilité supérieure tout en s'assurant que toutes ces responsabilités de niveau inférieur sont cohérentes entre elles.



Elies Jebri : elies.jebri@gmail.com

Cohésion et Couplage

- L'application de SoC implique deux processus : la **réduction du couplage** et **l'augmentation de la cohésion**.

- La cohésion est la mesure de la similarité d'un ensemble de tâches, du niveau de détails et de la localité.

  Par exemple, les fonctions drawCircle et drawTriangle sont suffisamment cohésives pour appartenir au même module responsable du dessin, et il semble naturel de mettre ces deux fonctions proches l'une de l'autre dans le code (forte similarité ~ forte cohésion).

- Le couplage, quant à lui, est la mesure de la dépendance de cette partie vis-à-vis du reste du système (faible dépendance ~ couplage lâche).

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Conseil

Un moyen rapide de se rappeler quel attribut doit être augmenté ou diminué :

- Le découplage est bon - nous devons donc viser un couplage lâche
- Le code cohérent est bon - nous devons viser une cohésion élevée

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Application du concept en IT

- En langage Java, la subdivision des composants est la suivante :
  - application
  - module (ie le concept introduit dans Java 9)
  - package
  - class
  - method
- Cela signifie qu'une application est composée d'un ou plusieurs module, que chaque module est composé d'un ou plus package, que chaque package est composé d'une ou plusieurs class, que chaque class est composée d'une ou plusieurs method.

- Bien entendu, cette division est déclinable à différentes échelles. Ainsi pour un système logiciel simplifié, cette division est :
  - système
  - sous-système
  - application
  - etc

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Avantages du couplage lâche et de la cohésion élevée

1. Meilleure clarté du code.
2. Meilleure réutilisabilité du code (principe DRY).
3. Meilleure testabilité.
4. Évolution plus rapide du projet.
5. Plus facile d'organiser le développement simultané

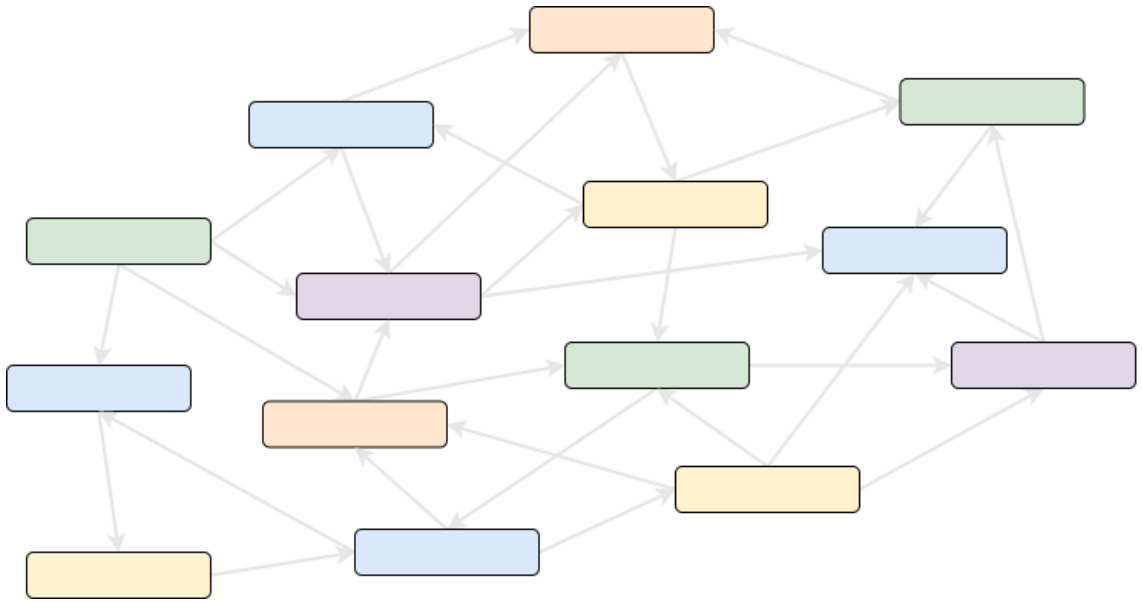
Elies Jebri

Elies Jebri : elies.jebri@gmail.com

SoC pour la conception du système

- Pour un grand nombre de modules avec des responsabilités distinctes et un objectif clair, nous devons encore définir une **stratégie** globale sur la façon dont les modules doivent **se référer les uns aux autres**.

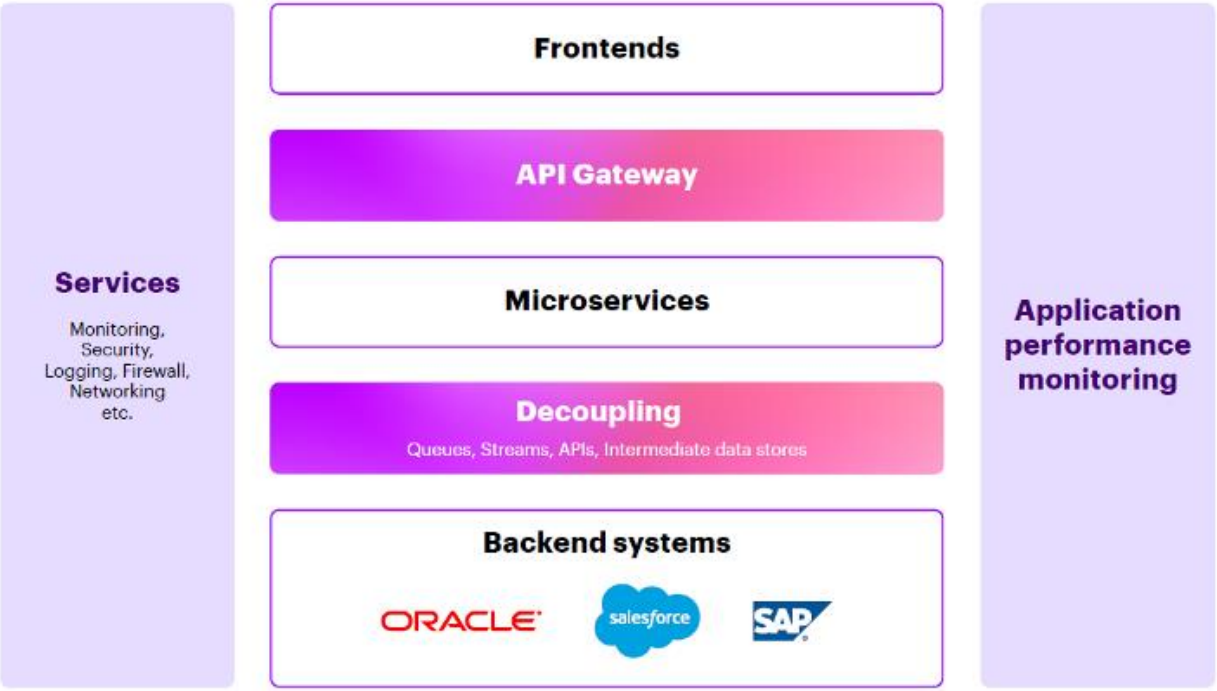
- Si nous n'introduisons pas cette stratégie, nous pouvons nous retrouver avec un système avec des relations enchevêtrées et des flux de données difficiles à suivre.



Elies Jebri : elies.jebri@gmail.com

Performance dans les architectures découplées

- Dans une architecture découplée, la performance existe au sein d'un écosystème complexe de nombreux composants en interaction.
- Il peut être difficile de déterminer exactement où se trouvent les goulots d'étranglement des performances.



Elies Jebri : elies.jebri@gmail.com

Performances
Interactions et chaînage des microservices

- Portez une attention particulière à la granularité et aux interactions entre les microservices.
- Les microservices trop volumineux peuvent être plus performants mais ne sont que des mini-monolithes réintroduits, annulant ainsi les avantages d'une architecture découplée.
- Des microservices trop petits entraîneront un trop grand nombre d'appels de service à service, ce qui ralentira les temps de réponse pour l'ensemble de la transaction commerciale de bout en bout.
- Si lors de la conception, par exemple, il s'avère qu'un microservice serait appelé plusieurs fois par seconde, une approche possible pourrait consister à convertir le microservice en une bibliothèque partagée, éliminant ainsi la latence d'appel hors processus.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Présentation de l'outil APM (Application Performance Management)

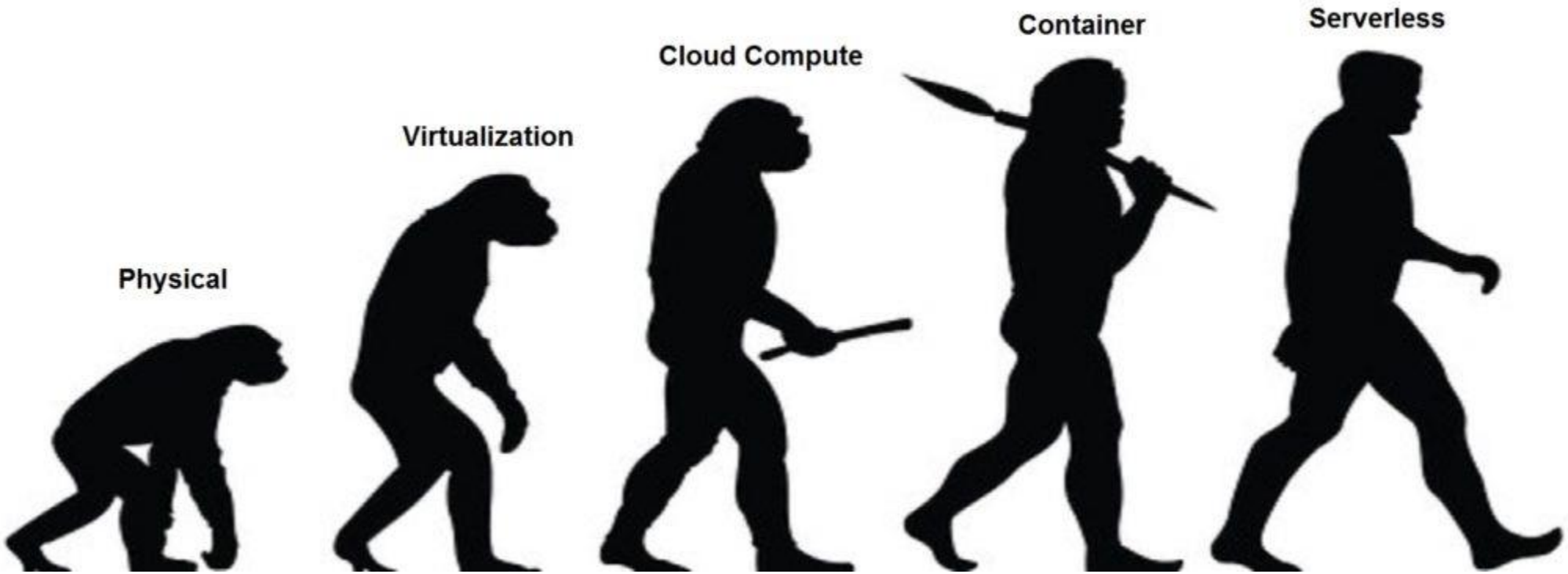
- Pour garder une trace des interactions des composants, la journalisation seule n'est pas suffisante.
- Il existe des outils APM qui injectent des en-têtes de message spéciaux dans les appels d'API, ce qui permet une véritable analyse des performances du système de bout en bout.
- Les outils APM disposent de plug-ins et d'agents de surveillance pour collecter de nombreuses métriques à partir de toutes les couches du système, du navigateur frontal aux bases de données principales et tout le reste.
- Le plus important est la capacité d'observer et d'analyser les performances du système de bout en bout, du navigateur aux bases de données principales.

- Par exemple, un outil APM fournira une vue graphique des interactions de microservices, le temps que chaque interaction a pris pour se terminer et une analyse détaillée des appels d'API spécifiques.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Et du côté des Ops !!!



Elies Jebri : elies.jebri@gmail.com

Ops c’est quoi?

- Ops est la constellation des compétences techniques, des pratiques et des valeurs culturelles de votre organisation pour la conception, la construction et la maintenance de systèmes, l'expédition de logiciels et la résolution de problèmes avec la technologie.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

L’évolution des architectures applicatives

- Que ce soit les évolutions matérielles, l’avènement de l’IoT ou encore les services proposés par les Cloud Service Providers.
- Le monde du développement logiciel n’échappe pas à cette tendance.
- Outre les nouveaux frameworks Web qui sortent plus vite que notre courbe d’apprentissage, les architectures applicatives elles aussi se voient repensées, remaniées.
- Il y a encore peu de temps, nous pensions tous qu’un bon vieux monolithe était “LA” solution simple, efficace et pas chère.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

L’évolution

- Avec l’émergence de la conteneurisation et du DevOps, un nouveau panel d’architectures a vu le jour.
- Nous avons ainsi vu les architectures Microservices.
- Mais comme toute nouvelle architecture, celle-ci venait avec son lot de contraintes.
- L’une d’entre elles est la gestion de l’infrastructure.
- Même si le DevOps et la conteneurisation ont apporté beaucoup dans cette problématique, ils ne l’ont pas résolue pour autant.
- Aujourd’hui, une nouvelle architecture fait parler d’elle dans le monde de l’IT, c’est le **Serverless**.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Qu’est-ce que le Serverless

- Applications qui dépendent de services tiers (Backend en tant que service ou «BaaS»).
- De nos jours la migration de nombreuses fonctionnalités côté FrontEnd nous a permis de supprimer nos besoins de serveurs “Always On”.
- Réduire le coût et la complexité opérationnels et ainsi payer uniquement les frais d’utilisations (bande passante, volume de stockage).
- Aussi connu comme **pay-as-you-go**.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

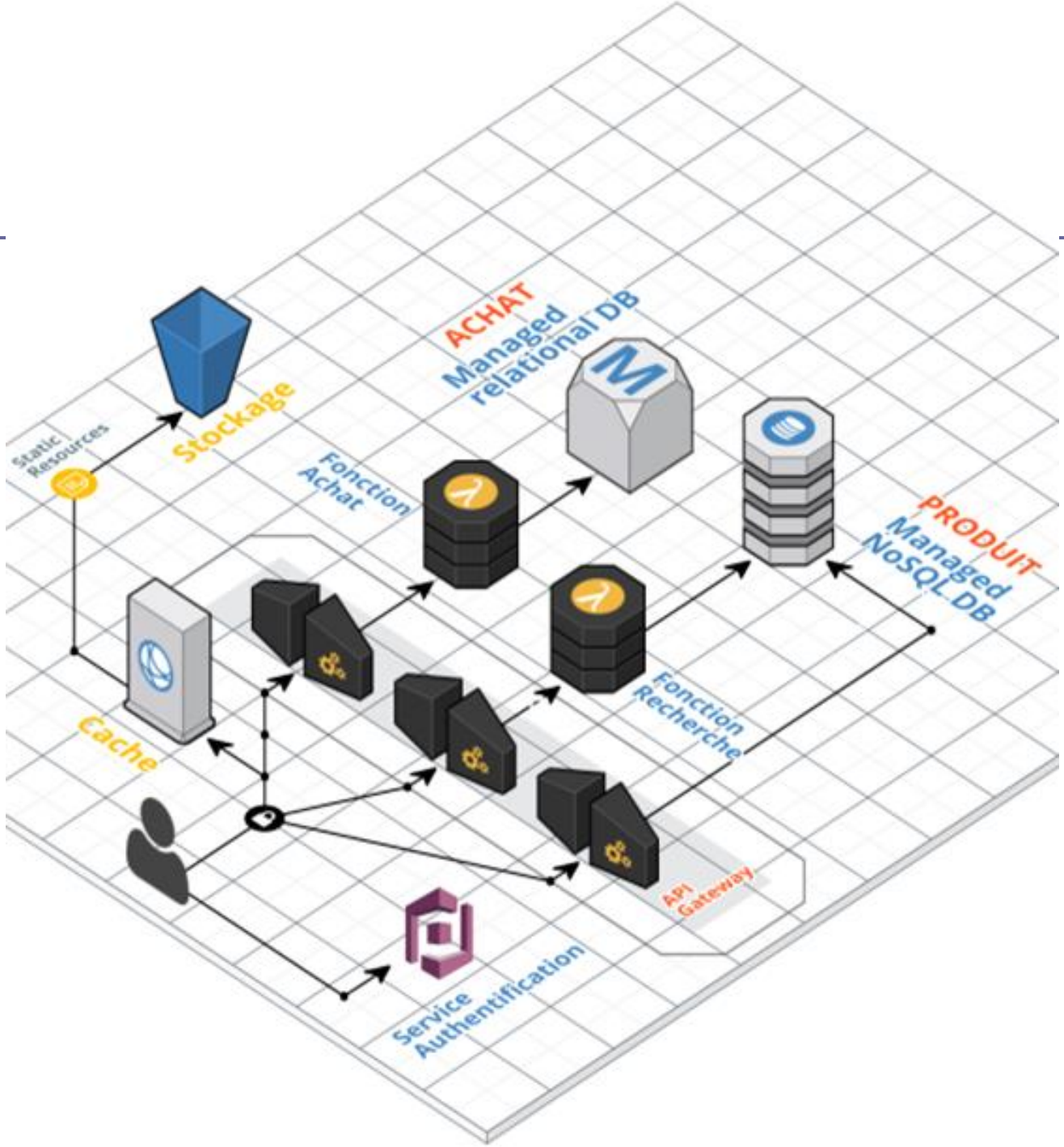
Qu’est-ce que le Serverless

- Serverless peut également symboliser des applications où une certaine quantité de logique serveur est toujours écrite par le développeur, mais contrairement aux architectures traditionnelles (exemple: Monolith), elle est exécutée dans des conteneurs stateless qui sont déclenchés par le biais d’événements, éphémère (uniquement une invocation) et entièrement géré par une 3rd party.
- Une façon de penser à ceci est le terme Function as a service ou FaaS.
- AWS Lambda est l’une des implémentations les plus populaires de FaaS à l’heure actuelle, mais il y en a d’autres (Google Functions).

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Exemple site e-commerce



Elies Jebri : elies.jebri@gmail.com

Site e-commerce

- Plus besoin d'héberger nos fichiers Web sur un serveur, un simple **bucket** et un outil de gestion du cache peuvent suffir.
- Plus de logique d'authentification mais un service tiers BaaS (Auth0, AWS Cognito, …).
- Par un BaaS, nous avons permis à un client d'accéder directement aux données (listes de produits), hébergées par un tiers (AWS DynamoDB, Firebase).
- Etablir des règles de sécurité liées aux profils utilisateurs et gérer les restrictions d'accès aux données.
- Changement des rôles des tiers : la logique est passée du serveur au client (exemple : suivi d'une session utilisateur, UX, lecture de données et affichage dans une vue utilisable.
- Certaines fonctionnalités sont à garder côté serveur, par exemple, si elles impliquent des calculs intensifs ou nécessitent l'accès à des quantités importantes de données (recherche de produits).
- Pour cette fonctionnalité, au lieu d'avoir un serveur toujours en cours d'exécution, nous pouvons implémenter une fonction FaaS qui répond aux requêtes HTTP via une API Gateway.
- Finalement, nous pouvons remplacer la fonctionnalité "Achat" par une autre fonction FaaS, en choisissant de la garder sur le côté serveur pour des raisons de sécurité.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

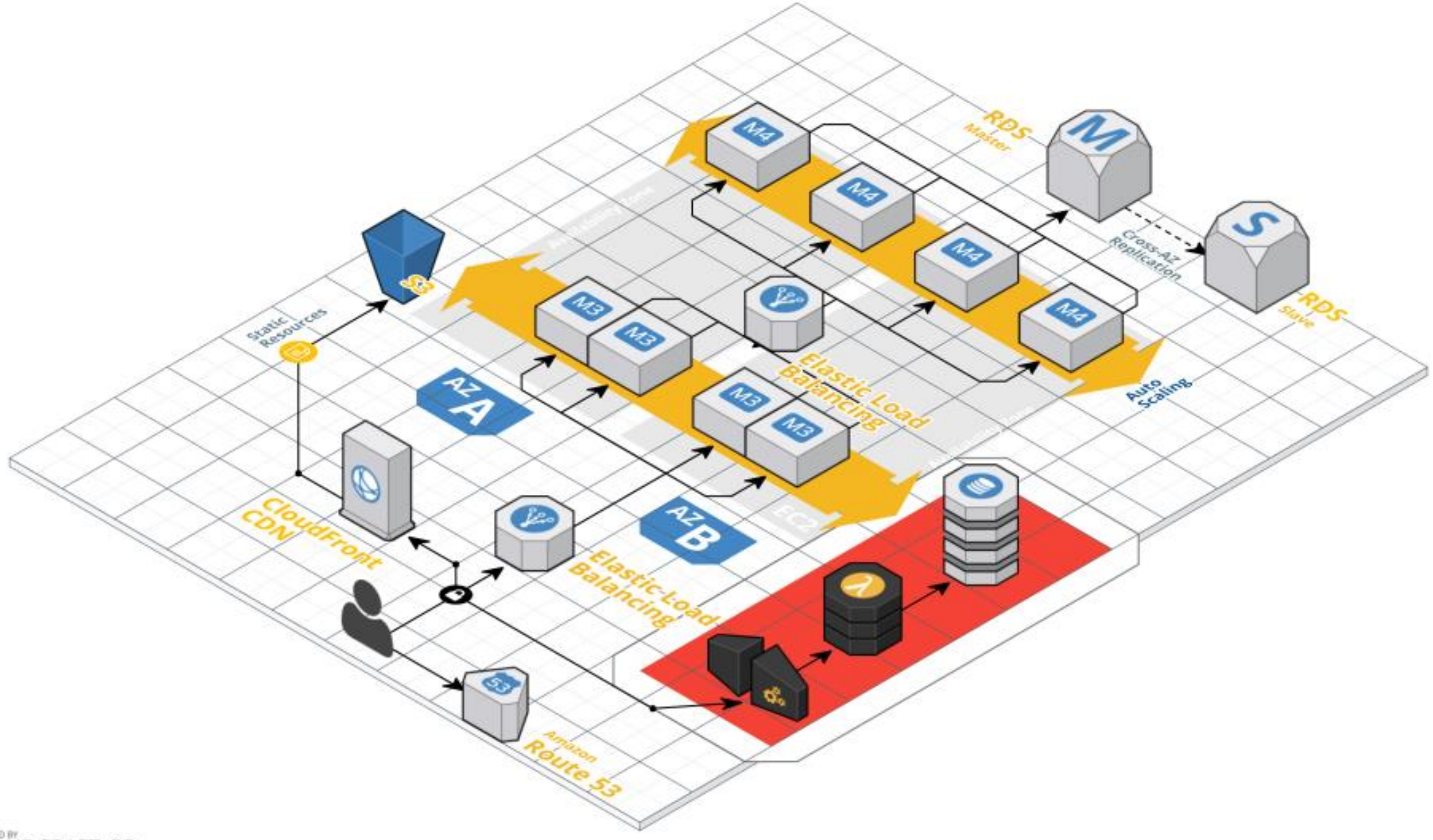
Application basée sur les événements (Event-driven)

- Un autre Use Case d'architecture Serverless est l'analyse et le traitement de données asynchrones.
- Prenons un cas que nous connaissons tous, le tracking d'activités des utilisateurs d'une application.
- Le besoin est d'arriver à connaître les activités précises d'un utilisateur sans impacter les performances de l'application.
- Nous pourrons ainsi déporter la logique permettant l'envoi/réception d'un événement de navigation et son enregistrement au sein d'une base de données de type BaaS.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com

Event-driven



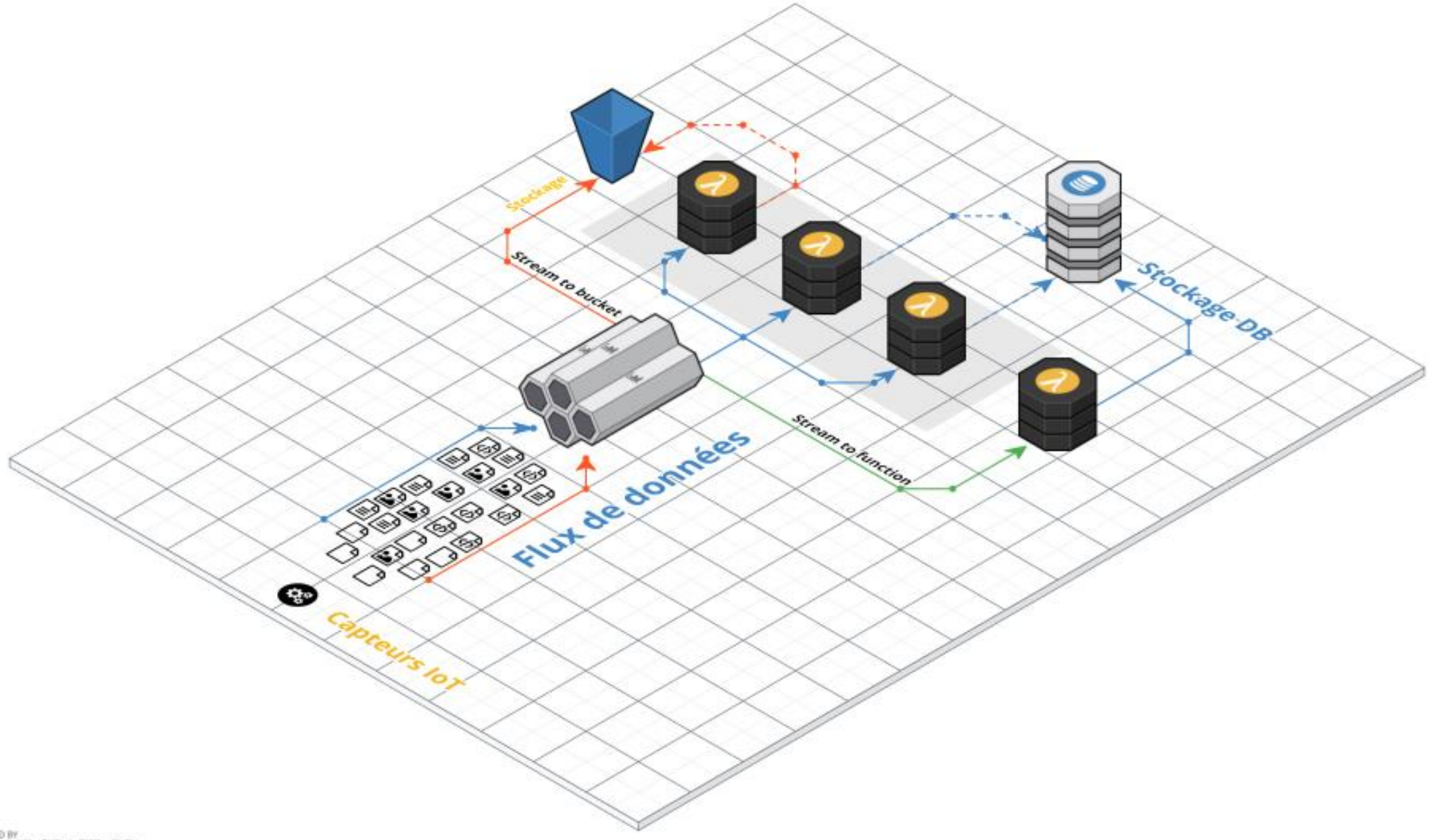
POWERED BY CLOUDCRAFT.CO

Exemple IoT

- Un autre exemple très en vogue de nos jours concerne l'IoT et son envoi d'informations effectué par des milliers de capteurs.
- Aucun besoin d'une communication synchrone entre la partie serveur et les capteurs, en revanche l'objectif est la récupération des informations, son stockage et son analyse.
- Un exemple de ce Use Case est la maintenance prédictive.
- Après la récupération des données brutes issues des capteurs, des algorithmes vont les analyser et pouvoir conjecturer et "prédire" une panne ou un besoin de maintenance d'un équipement.

Elies Jebri

Elies Jebri : elies.jebri@gmail.com



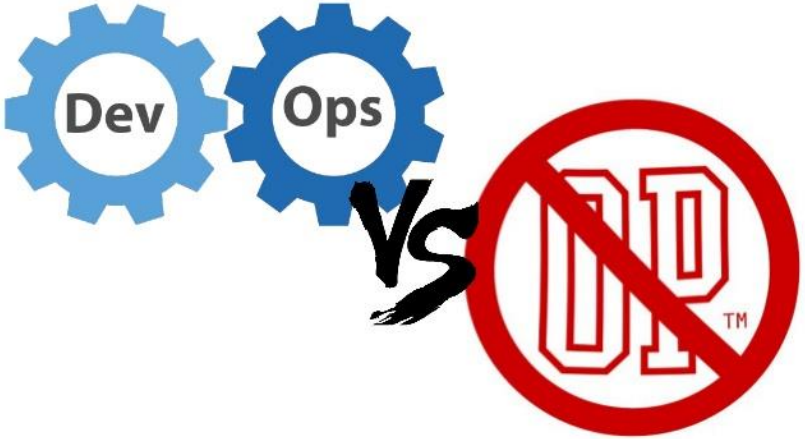
IoT

POWERED BY CLOUDCRAFT.CO

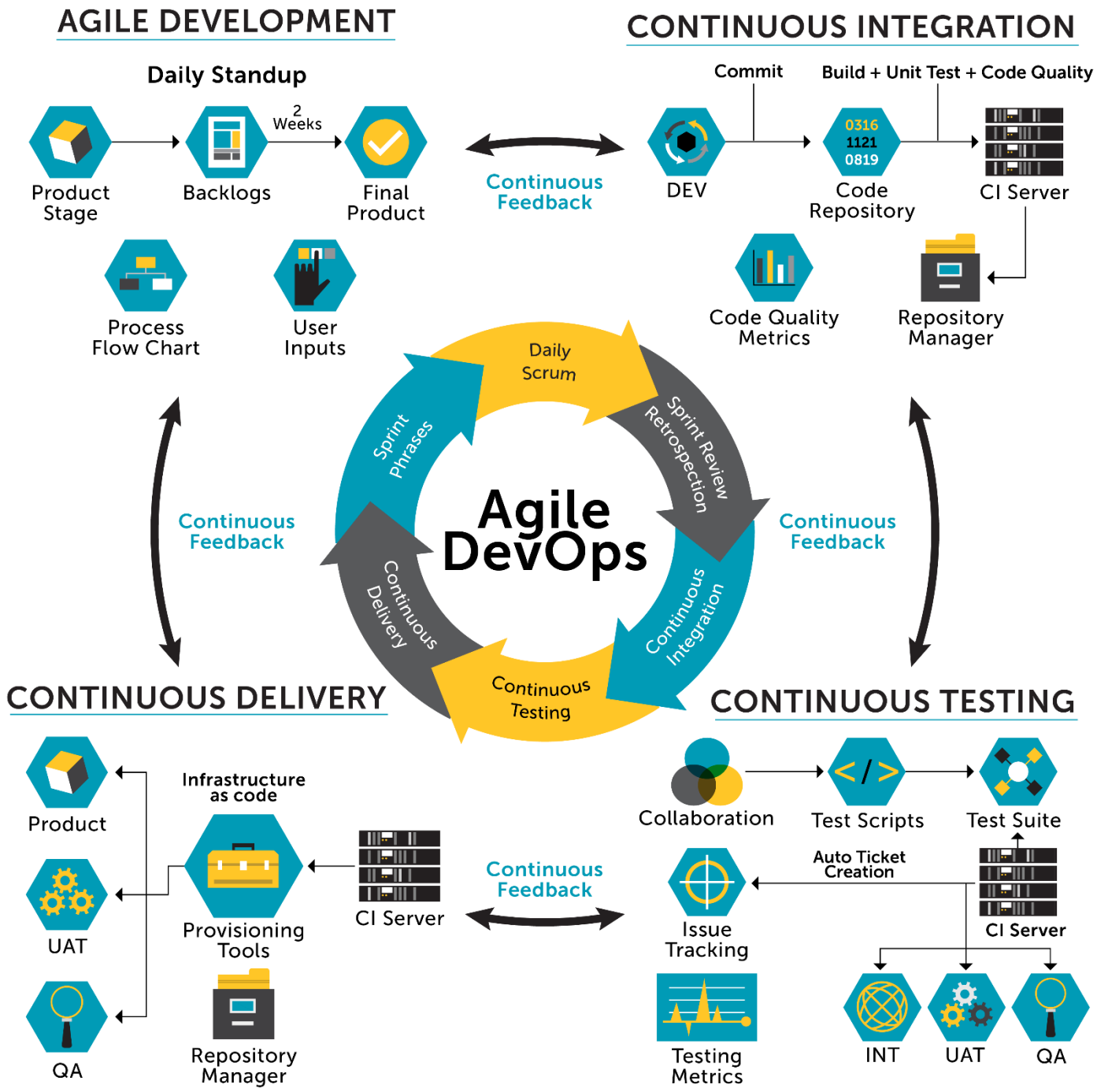
NoOps !!!

- NoOps (No Operations) – l'absence de personnel d'exploitation – est un concept selon lequel un environnement informatique a atteint un niveau d'automatisation et de virtualisation suffisant, par rapport à son infrastructure sous-jacente, pour que plus aucune équipe interne ne soit nécessaire à l'administration de l'infrastructure logicielle.

Elies Jebri



Elies Jebri : elies.jebri@gmail.com



Elies Jebri : elies.jebri@gmail.com

Elies Jebri : elies.jebri@gmail.com